

UNIVERSIDADE ESTADUAL DE SANTA CRUZ
DEPARTAMENTO DE CIÊNCIAS EXATAS E TECNOLÓGICAS - DCET
CIÊNCIA DA COMPUTAÇÃO

KAYQUE COSTA MOREIRA

SECRETARIA - DESENVOLVIMENTO DE UM ASSISTENTE PESSOAL INTELIGENTE COM FOCO EM AGENDA E GERENCIAMENTO DE SENHAS

ILHÉUS
2025

KAYQUE COSTA MOREIRA

SECRETARIA - DESENVOLVIMENTO DE UM ASSISTENTE PESSOAL INTELIGENTE COM FOCO EM AGENDA E GERENCIAMENTO DE SENHAS

Trabalho de Conclusão de Curso submetido à Universidade Estadual de Santa Cruz, como requisito parcial para obtenção do grau de Bacharel em Ciência de Computação.

Orientador: Prof. Dr. Álvaro Vinícius de Souza Coêlho

ILHÉUS
2025

KAYQUE COSTA MOREIRA

SECRETARIA - DESENVOLVIMENTO DE UM ASSISTENTE PESSOAL INTELIGENTE COM FOCO EM AGENDA E GERENCIAMENTO DE SENHAS

Trabalho de Conclusão de Curso submetido à Universidade Estadual de Santa Cruz, como requisito parcial para obtenção do grau de Bacharel em Ciência de Computação.

Ilhéus, 09, de julho de 2025

Prof. Dr. Álvaro Vinícius de Souza Coêlho
UESC/DCET

Prof. Dr. Paulo Eduardo Ambrósio
UESC/DCET

Prof. Me. Jauberth Weyll Abijaude
UESC/DCET

AGRADECIMENTOS

A Deus

A meu pai, minha mãe e meu irmão

A meus avós, meus tios, meus primos

Aos meus amigos

Ao meu orientador

Ao CEPEDI por proporcionar um ambiente propício para o meu desenvolvimento profissional

À UESC por proporcionar toda a estrutura necessária para a graduação

RESUMO

Este trabalho descreve o desenvolvimento da "secretarIA", uma aplicação web que visa integrar gerenciamento de agenda, cofre de senhas e interação conversacional via IA para otimizar a produtividade pessoal. Dada a fragmentação de ferramentas digitais e os avanços em LLMs, a secretarIA foi projetada para centralizar tarefas e oferecer uma interface mais natural. O desenvolvimento seguiu uma abordagem iterativa, utilizando Next.js, React, Node.js, MongoDB, LangChain e a API da OpenAI. A arquitetura foi baseada em camadas, visando manutenibilidade e escalabilidade. A avaliação do protótipo incluiu análise funcional, avaliação heurística de usabilidade e análise da interação com a IA. Os resultados indicam sucesso na implementação das funcionalidades básicas e potencial na interação via IA, apesar das limitações identificadas na compreensão de linguagem natural complexa. O trabalho contribui com um estudo de caso sobre a integração de LLMs em IPAs especializados e discute desafios de UX, segurança e privacidade

Palavras-chave: Assistente Pessoal Inteligente. Modelo de Linguagem Grande. LangChain. Gerenciamento de Agenda. Gerenciador de Senhas. Next.js. Experiência do Usuário.

ABSTRACT

This work describes the development of "secretarIA", a web application aimed at integrating calendar management, password vault, and conversational AI interaction to optimize personal productivity. Given the fragmentation of digital tools and advances in LLMs, secretarIA was designed to centralize tasks and offer a more natural interface. Development followed an iterative approach using Next.js, React, Node.js, MongoDB, LangChain, and the OpenAI API. The architecture was layer-based, focusing on maintainability and scalability. Prototype evaluation included functional analysis, heuristic usability assessment, and AI interaction analysis. Results indicate successful implementation of core features and potential in AI interaction, despite identified limitations in understanding complex natural language. The work contributes a case study on integrating LLMs into specialized IPAs and discusses UX, security, and privacy challenges.

Keywords: Intelligent Personal Assistant. Large Language Model. LangChain. Calendar Management. Password Manager. Next.js. User Experience.

LISTA DE ABREVIATURAS E SIGLAS

AES	Padrão de Criptografia Avançada
API	Interface de Programação de Aplicação
ASR	Reconhecimento Automático de Fala
CRUD	Criar, Ler, Atualizar e Deletar
CSR	Renderização do Lado do Cliente
CSS	Folhas de Estilo em Cascata
CUI	Interface de Chat
E2EE	Criptografia de Ponta a Ponta
GPT	Transformador Pré-treinado Generativo
HTML	Linguagem de Marcação de Hipertexto
HTTP	Protocolo de Transferência de Hipertexto
HTTPS	Protocolo de Transferência de Hipertexto Seguro
IA	Inteligência Artificial
IPA	Assistente Pessoal Inteligente
IR	Recuperação de Informação
IV	Initialization Vector
JWT	Token Web JSON
LLM	Modelo de Linguagem Grande
LGPD	Lei Geral de Proteção de Dados Pessoais
ML	Aprendizado de Máquina
NLG	Geração de Linguagem Natural
NLU	Compreensão da linguagem natural
NoSQL	Não Apenas SQL
OAuth	Autorização Aberta
ODM	Documento-Objeto Mapeador
OWASP	Projeto Aberto de Segurança de Aplicativos da Web
PLN	Processamento de Linguagem Natural
REST	Transferência de Estado Representacional
RF	Requisito Funcional
RNF	Requisito Não Funcional

SSR	Renderização do Lado do Servidor
SQL	Linguagem de Consulta Estruturada
TTS	Conversão de Texto para Fala
UI	Interface do Usuário
URI	Identificador Uniforme de Recursos
URL	Recurso Uniforme Localizador
UX	Experiência do Usuário
VUI	Interface de Usuário de Voz
XAI	Inteligência Artificial Explicável
XSS	Script Entre Sites

LISTA DE FIGURAS

Figura 1 – Diagrama da arquitetura de três camadas do SecretarIA, destacando os fluxos de dados, autenticação e integração com serviços externos. 45
Figura 2 – Tela do dashboard do SecretarIA, destacando a integração dos módulos principais 46
Figura 3 – Estrutura das coleções no MongoDB Atlas 48
Figura 4 – Tela de login do SecretarIA 49
Figura 5 – Visualização do calendário e modal de evento 50
Figura 6 – Lista de senhas com campos mascarados 52
Figura 7 – Interação do usuário com o chat inteligente 54
Figura 8 – Exemplo de notificação de e-mail enviado pelo sistema 55
Figura 9 – Linha do tempo do desenvolvimento do SecretarIA 58

LISTA DE TABELAS

Tabela 1 – Requisitos Funcionais 41
Tabela 2 – Requisitos Não Funcionais 43
Tabela 3 – Avaliação dos Requisitos Funcionais 61

SUMÁRIO

1 INTRODUÇÃO 14
1.0.1 Problema 14
1.0.2 Justificativa 15
1.1 Objetivos 16
1.1.1 Objetivo Geral 16
1.1.2 Objetivos Específicos 16
2 MATERIAIS E MÉTODOS 18
2.1 Arquitetura do Sistema 18
2.2 Padrões de Projeto 18
2.3 Tecnologias Utilizadas 19
2.3.1 Next.js™ e React™ 19
2.3.2 MongoDB™ 20
2.4 Ferramentas para Suporte à IA Generativa 20
2.4.1 O que é IA Generativa e LLMs? 20
2.4.2 LangChain™: Orquestração de Fluxos de Conversa 20
2.4.3 OpenAI™: Modelos de Linguagem Avançados 21
2.4.4 Resumo 22
2.4.5 NextAuth™ e Nodemailer™ 22
2.4.6 Prisma ORM™ e Mongoose™ 24
2.4.7 Tailwind CSS™ 26
2.5 Ferramentas de Apoio ao Desenvolvimento 28
2.6 Segurança e Privacidade 30
2.7 Estratégias de Testes 30
2.8 Metodologia de Desenvolvimento 32
3 METODOLOGIA 33
3.1 Abordagem Metodológica 33
3.2 Adaptações Metodológicas para Projeto Individual de TCC . . . 33
3.3 Fases do Projeto 34
3.3.1 Pesquisa e Definição 34
3.3.2 Configuração Inicial 35
3.3.3 Desenvolvimento dos Módulos Principais 35

3.3.4 Refinamento e Testes . . . 36
3.3.5 Documentação e Redação Final . . . 37
3.4 Ferramentas e Tecnologias . . . 37
3.4.1 JavaScript™/TypeScript™ . . . 37
3.4.2 Next.js™ e React™ . . . 37
3.4.3 Tailwind CSS™ . . . 38
3.4.4 MongoDB Atlas™ . . . 38
3.4.5 Prisma ORM™/Mongoose™ . . . 38
3.4.6 NextAuth.js™ . . . 38
3.4.7 LangChain™ e OpenAI™ (GPT-3.5/4) . . . 39
3.4.8 Nodemailer™ . . . 39
3.4.9 Visual Studio Code™, Git™ e GitHub™ . . . 39
3.5 Avaliação do Protótipo . . . 39
4 DESENVOLVIMENTO . . . 41
4.1 Requisitos do Sistema . . . 41
4.1.1 Requisitos Funcionais . . . 41
4.1.2 Requisitos Não Funcionais . . . 43
4.1.3 Visão Geral do Sistema . . . 44
4.1.4 Arquitetura e Estrutura do Projeto . . . 44
4.1.5 Detalhamento do Frontend . . . 45
4.1.6 Backend e Integração com Serviços . . . 46
4.1.7 Persistência e Segurança dos Dados . . . 47
4.1.8 Funcionalidades-Chave do SecretarIA . . . 48
4.1.9 Autenticação e Controle de Acesso . . . 48
4.1.10 Módulo Calendário . . . 49
4.1.11 Gerenciador de Senhas . . . 50
4.1.12 Chat Inteligente . . . 52
4.1.13 Notificações por E-mail . . . 55
4.1.14 Segurança, Privacidade e Boas Práticas . . . 56
4.1.15 Desafios, Aprendizados e Evolução . . . 57
5 RESULTADOS . . . 60
5.0.1 Apresentação da Aplicação Final . . . 60
5.0.2 Avaliação do Cumprimento dos Requisitos . . . 61

5.0.3 Análise da Interação Conversacional com IA 61
5.0.4 Avaliação de Usabilidade 63
5.0.5 Discussão sobre Segurança e Privacidade 64
5.0.6 Considerações Finais dos Resultados 64
6 CONCLUSÃO 66
REFERÊNCIAS 68

1 INTRODUÇÃO

O avanço acelerado das tecnologias digitais e a popularização dos assistentes virtuais têm transformado a forma como as pessoas organizam suas rotinas e gerenciam informações pessoais (MAEDCHE et al., 2019). Em um cenário marcado pelo aumento do volume de compromissos, senhas e dados sensíveis, torna-se cada vez mais necessário contar com soluções inteligentes que promovam não apenas a praticidade, mas também a segurança e a integração de diferentes funcionalidades no cotidiano do usuário (MOLS et al., 2021).

Apesar da disponibilidade de aplicativos de agenda, gerenciadores de senhas e sistemas de notificações, muitos desses recursos ainda operam de maneira isolada, dificultando uma visão unificada e eficiente das tarefas diárias. Além disso, questões relacionadas à privacidade e à proteção de dados têm ganhado destaque, especialmente diante do crescimento de ataques cibernéticos e do uso indevido de informações pessoais (ISLAM et al., 2022).

Usuários frequentemente relatam esquecer compromissos importantes, anotar senhas em locais inseguros ou recorrer a métodos improvisados para gerenciar suas tarefas, o que demonstra a necessidade de uma solução centralizada, inteligente e segura (ISLAM; CHAUDHRY, 2022).

Diante desse contexto, o desenvolvimento de soluções que aliem organização pessoal, segurança da informação e usabilidade se apresenta como um desafio relevante tanto do ponto de vista tecnológico quanto social. O **SecretarIA** surge como uma proposta inovadora ao integrar funcionalidades de assistente inteligente, gerenciamento de agenda, armazenamento seguro de senhas e envio automatizado de notificações, utilizando tecnologias modernas e práticas robustas de segurança.

1.0.1 Problema

Atualmente, é comum o uso de múltiplos sistemas para organização pessoal, como agendas eletrônicas, gerenciadores de senhas e aplicativos de lembre-

tes (ZHANG et al., 2019). No entanto, a ausência de integração entre essas ferramentas ainda resulta em experiências fragmentadas e pouco eficientes para o usuário.

Recentemente, a incorporação de soluções baseadas em inteligência artificial (IA) em sistemas de apoio à organização pessoal tem mostrado grande potencial para transformar esse cenário. A IA permite automatizar tarefas repetitivas, personalizar recomendações, interpretar comandos em linguagem natural e antecipar necessidades do usuário, tornando as interações mais inteligentes e proativas (ISLAM et al., 2022).

Apesar desse potencial, muitos sistemas atuais ainda não exploram de forma adequada os recursos de IA, limitando-se a funcionalidades básicas e sem oferecer uma experiência realmente integrada(MAYER et al., 2023). Isso dificulta o aproveitamento pleno das vantagens que a IA pode proporcionar, como a centralização de informações, a automação de notificações e a melhoria da segurança no gerenciamento de dados sensíveis.

Nesse contexto, o problema central deste trabalho consiste em identificar como desenvolver um sistema de organização pessoal que utilize inteligência artificial de forma efetiva, integrando funcionalidades como agenda, gerenciador de senhas e notificações inteligentes, com foco em praticidade, segurança e personalização da experiência do usuário.

1.0.2 Justificativa

A crescente complexidade das rotinas pessoais e profissionais, aliada à necessidade de proteger informações sensíveis, evidencia a importância de ferramentas que unifiquem a gestão de compromissos e dados de forma segura e acessível (MOLS et al., 2021). Muitos usuários enfrentam dificuldades ao utilizar múltiplos aplicativos para diferentes finalidades, o que pode resultar em perda de produtividade, esquecimentos e exposição a riscos de segurança (ISLAM; CHAUDHRY, 2022).

Além disso, a falta de integração entre sistemas de agenda, gerenciadores de senhas e assistentes virtuais limita o potencial dessas tecnologias em oferecer uma

experiência realmente personalizada e eficiente. O SecretarIA busca preencher essa lacuna ao propor uma solução que centraliza essas funcionalidades, promovendo maior praticidade, segurança e autonomia ao usuário.

O projeto também se justifica pela relevância acadêmica e tecnológica, ao explorar temas atuais como inteligência artificial aplicada à organização pessoal, experiência do usuário (UX) e segurança da informação, contribuindo para o avanço do conhecimento e o desenvolvimento de soluções inovadoras na área.

Diante desse contexto, esta pesquisa propõe o desenvolvimento de um sistema robusto e acessível, alinhado às necessidades de pessoas que buscam otimizar sua rotina, centralizar informações sensíveis e contar com o apoio de um assistente inteligente para aumentar a produtividade e a segurança no dia a dia.

1.1 Objetivos

1.1.1 Objetivo Geral

Desenvolver um assistente inteligente, denominado SecretarIA, capaz de integrar funcionalidades de organização pessoal, gerenciamento seguro de senhas e envio automatizado de notificações, com foco em usabilidade e segurança da informação.

1.1.2 Objetivos Específicos

- Implementar um sistema de agenda que permita o cadastro, edição e visualização de compromissos de forma intuitiva, utilizando IA generativa para facilitar a interação do usuário;
- Desenvolver um gerenciador de senhas seguro, utilizando criptografia para armazenamento dos dados sensíveis e IA generativa para auxiliar o usuário na criação e recuperação de senhas de maneira amigável;
- Integrar um mecanismo de notificações automáticas por e-mail para lembrete de eventos e atividades, empregando IA generativa para personalizar as mensagens e torná-las mais compreensíveis;

- Aplicar práticas de segurança da informação em todas as etapas do desenvolvimento, contando com o apoio da IA generativa para identificar possíveis vulnerabilidades e sugerir melhorias de forma proativa;
- Avaliar a experiência do usuário por meio de testes de usabilidade e feedbacks, utilizando IA generativa para analisar respostas e propor ajustes que tornem o sistema mais intuitivo;
- Documentar todo o processo de desenvolvimento, destacando desafios, soluções adotadas e resultados obtidos, com suporte da IA generativa para organizar e apresentar as informações de forma clara e acessível.

2 MATERIAIS E MÉTODOS

Este capítulo apresenta de forma detalhada as tecnologias, ferramentas, padrões de projeto e métodos empregados no desenvolvimento do SecretarIA, com a devida justificativa para cada escolha, sempre alinhada aos objetivos e requisitos do sistema.

2.1 Arquitetura do Sistema

O SecretarIA foi desenvolvido adotando uma arquitetura de três camadas, composta por apresentação, lógica de negócios e persistência de dados. Essa abordagem modular é amplamente reconhecida na engenharia de software por facilitar a manutenção, promover a escalabilidade e garantir a separação de responsabilidades entre os diferentes componentes do sistema (BASS et al., 2012; FOWLER, 2002).

A camada de apresentação foi implementada utilizando Next.js™ e React™, proporcionando ao usuário uma interface responsiva, moderna e de fácil utilização. Já a camada de lógica de negócios é composta pelas API Routes do Next.js™, responsáveis pelo processamento das regras do sistema e pela integração com serviços externos, como provedores de autenticação e APIs de inteligência artificial.

Por fim, a camada de persistência de dados utiliza o banco de dados MongoDB™, que armazena informações dos usuários, eventos, senhas e histórico de interações de forma eficiente e segura.

2.2 Padrões de Projeto

Para promover o desacoplamento entre os módulos e facilitar a manutenção, foi adotado o padrão *Observer* (GAMMA et al., 1994). O Observer é amplamente utilizado em sistemas reativos e desacoplados, pois permite que diferentes partes do sistema sejam notificadas automaticamente sobre mudanças de estado, sem depender de uma integração direta ou rígida entre os componentes.

Esse padrão estabelece uma relação na qual um objeto (o "sujeito") mantém uma lista de dependentes (os "observadores") e os notifica sempre que ocorre uma alteração relevante. Os observadores podem, então, reagir a esses eventos de forma autônoma, sem que o sujeito precise conhecer detalhes de suas implementações, promovendo flexibilidade e extensibilidade no sistema.

No contexto do SecretarIA, o padrão Observer é especialmente adequado para o gerenciamento de notificações automáticas e atualização de interfaces. Por exemplo, quando um novo compromisso é criado ou alterado no calendário, módulos observadores — como o responsável pelo envio de notificações por e-mail ou pela atualização da interface do usuário — são imediatamente informados sobre o evento, podendo executar suas ações de forma independente. Dessa maneira, o sistema garante que as mensagens de notificação sejam disparadas de modo eficiente e desacoplado, tornando a experiência do usuário mais intuitiva e responsiva.

2.3 Tecnologias Utilizadas

2.3.1 Next.js™ e React™

O framework Next.js™, construído sobre o React™, foi selecionado devido à sua flexibilidade e ao suporte nativo a múltiplas estratégias de renderização, incluindo Server-Side Rendering (SSR), Static Site Generation (SSG) e Client-Side Rendering (CSR). Essa variedade de abordagens permite otimizar tanto o desempenho quanto a experiência do usuário, adaptando-se a diferentes demandas do sistema (VERCEL, 2024).

A componentização proporcionada pelo React™ facilita o desenvolvimento de interfaces reutilizáveis, modulares e de fácil manutenção, promovendo a escalabilidade e a padronização do código (PLATFORMS, 2024). Além disso, a integração eficiente entre frontend e backend oferecida pelo Next.js™ simplifica a implementação de rotas de API e a comunicação com serviços externos, tornando a arquitetura do SecretarIA mais coesa e robusta.

2.3.2 MongoDB™

O banco de dados NoSQL MongoDB™ foi adotado devido à sua capacidade de armazenar documentos flexíveis e escalar horizontalmente. Isso atende à necessidade de gerenciar diferentes tipos de dados do SecretarIA, como eventos de calendário, senhas criptografadas e registros de usuários, permitindo alterações no esquema sem grandes impactos na aplicação.

2.4 Ferramentas para Suporte à IA Generativa

Para que o SecretarIA seja capaz de compreender comandos, dialogar e auxiliar o usuário de maneira inteligente, foram empregadas ferramentas modernas de inteligência artificial generativa, com destaque para LangChain™ e OpenAI™. Esta seção apresenta os conceitos fundamentais e justifica a escolha dessas tecnologias.

2.4.1 O que é IA Generativa e LLMs?

A inteligência artificial generativa refere-se a sistemas capazes de criar novos conteúdos — como textos, imagens ou músicas — a partir de exemplos e instruções fornecidos pelo usuário (BOMMASANI et al., 2021). O avanço recente dessas soluções se deve, em grande parte, ao desenvolvimento dos chamados Large Language Models (LLMs), modelos de linguagem de larga escala treinados com grandes volumes de dados textuais. LLMs, como o GPT-3 e GPT-4, são capazes de compreender linguagem natural, gerar respostas contextuais e executar tarefas complexas de processamento de texto (BROWN et al., 2020).

No SecretarIA, a IA generativa é utilizada para interpretar comandos, gerar respostas automáticas e auxiliar o usuário em tarefas relacionadas à organização pessoal.

2.4.2 LangChain™: Orquestração de Fluxos de Conversa

O LangChain™ é uma biblioteca projetada para facilitar a integração, o gerenciamento de contexto e a orquestração de fluxos conversacionais com LLMs (LANGCHAIN, 2024). Ele permite armazenar o histórico de interações, montar prompts otimizados e

conectar diferentes modelos de IA de maneira flexível, funcionando como um “maestro” que organiza todo o fluxo da conversa.

No SecretarIA, o LangChain™ é responsável por:

- Gerenciar o histórico das conversas, tornando o diálogo mais natural e contextualizado;
- Construir prompts adequados antes de enviar perguntas ao modelo de IA;
- Integrar-se facilmente a diferentes provedores de LLMs, como a OpenAI™;
- Adaptar e entregar as respostas ao usuário, conforme necessário.

Exemplo prático: Se o usuário digita: “Lembre-me de pagar a conta amanhã”, o LangChain™:

1. Analisa o histórico de conversas para contexto;
2. Monta um prompt claro: “Usuário deseja um lembrete para pagar uma conta amanhã”;
3. Envia para a IA (OpenAI™);
4. Recebe a resposta, por exemplo: “Lembrete criado! Você será avisado amanhã para pagar a conta”;
5. Mostra isso para o usuário.

2.4.3 OpenAI™: Modelos de Linguagem Avançados

A OpenAI™ desenvolveu modelos de linguagem de última geração, como o GPT-3 e GPT-4 (OPENAI, 2024; BROWN et al., 2020), capazes de interpretar comandos em linguagem natural, responder perguntas, resumir informações e auxiliar na redação de textos.

No contexto do SecretarIA, a OpenAI™ é utilizada para:

- Gerar respostas automáticas para dúvidas do usuário;
- Auxiliar na criação de lembretes, resumos e notificações personalizadas;
- Interpretar comandos livres, permitindo uma interação flexível e intuitiva.

Fluxo visual simplificado:

Usuário → LangChain™ → OpenAI™ → LangChain™ → Usuário

Exemplo de código (simplificado):

```
// Exemplo em JavaScript/TypeScript
const resposta = await openai.createChatCompletion({
  model: "gpt-4",
  messages: [
    { role: "system", content: "Você é um assistente de agenda." },
    { role: "user", content: "Lembre-me de pagar a conta amanhã." }
  ]
});
console.log(resposta.data.choices[0].message.content);
```

2.4.4 Resumo

A combinação entre LangChain™ (LANGCHAIN, 2024) e OpenAI™ (OPENAI, 2024) permite que o SecretarIA ofereça uma experiência conversacional avançada, capaz de compreender comandos em linguagem natural, manter o contexto das interações e executar ações de apoio à organização pessoal do usuário.

2.4.5 NextAuth™ e Nodemailer™

A segurança e a comunicação eficiente com o usuário são aspectos fundamentais no SecretarIA. Para atender a esses requisitos, foram adotadas duas ferramentas principais: NextAuth™ para autenticação e gerenciamento de sessões, e Nodemailer™ para o envio automatizado de e-mails.

O NextAuth™ destaca-se por sua flexibilidade e robustez na implementação de autenticação segura em aplicações Next.js™. Ele oferece suporte a múltiplos provedores de login (como Google™, GitHub™ e autenticação por e-mail), gerenciamento eficiente de sessões e integração transparente com o backend da aplicação, além de utilizar padrões modernos de segurança e criptografia (NEXTAUTH.JS, 2024). Isso

garante que apenas usuários autenticados possam acessar e modificar informações sensíveis, como compromissos e senhas.

O Nodemailer™ é uma biblioteca amplamente utilizada para envio de e-mails em aplicações Node.js™. Sua adoção no SecretarIA permite a automação de notificações, como lembretes de eventos, confirmações de cadastro e alertas de segurança. A ferramenta se destaca pela facilidade de integração com diversos serviços de e-mail, capacidade de personalização das mensagens e confiabilidade no envio (NODEMAILER, 2024). Dessa forma, o sistema mantém o usuário sempre informado, reforçando a transparência e a segurança das operações.

NextAuth™: Autenticação Segura e Flexível

O NextAuth™ permite implementar login seguro de forma simples e flexível, suportando múltiplos provedores. Seu funcionamento consiste em:

- O usuário acessa a tela de login e escolhe um método de autenticação (por exemplo, Google™ ou e-mail).
- O NextAuth™ gerencia todo o fluxo de autenticação, redirecionando o usuário para o provedor escolhido e, após o login, retornando ao sistema já autenticado.
- As sessões dos usuários são protegidas por tokens seguros, evitando acessos não autorizados.

Entre as principais vantagens destacam-se:

- **Segurança:** Utiliza padrões modernos de autenticação e criptografia.
- **Flexibilidade:** Integração fácil com diversos provedores de login.
- **Facilidade de uso:** O usuário pode escolher o método de login mais conveniente.

No SecretarIA, o NextAuth™ garante que apenas usuários autenticados possam acessar e modificar dados sensíveis, além de facilitar o cadastro e acesso de novos usuários.

Nodemailer™: Envio Automático de E-mails

O Nodemailer™ permite o envio programático de e-mails diretamente pelo código do sistema. Sua utilização no SecretarIA abrange:

- Disparo automático de e-mails quando eventos relevantes ocorrem (como cadastro de novo compromisso ou alteração de senha).
- Envio de lembretes de eventos, confirmações de cadastro e alertas de segurança.

As principais vantagens do Nodemailer™ incluem:

- **Automação:** E-mails são enviados automaticamente, sem intervenção manual.
- **Personalização:** O conteúdo das mensagens pode ser adaptado para cada usuário e situação.
- **Confiabilidade:** Integra-se facilmente com diversos serviços de e-mail (Gmail, Outlook, etc.).

No SecretarIA, o Nodemailer™ assegura que o usuário receba lembretes automáticos de compromissos, notificações de mudanças de senha e confirmações de cadastro, promovendo segurança e transparência.

Fluxo resumido de integração:

1. O usuário faz login via NextAuth™.
2. Ao criar ou editar um compromisso, o sistema utiliza o Nodemailer™ para enviar um lembrete por e-mail.
3. O usuário recebe a notificação e pode se organizar melhor.

2.4.6 Prisma ORM™ e Mongoose™

O acesso e a manipulação de dados no SecretarIA são realizados de forma eficiente e segura graças ao uso combinado do Prisma ORM™ e do Mongoose™. Embora ambas as ferramentas facilitem o trabalho com o banco de dados MongoDB™, cada uma apresenta características e vantagens específicas que justificam sua adoção conjunta.

Em muitos projetos, a utilização de apenas uma dessas ferramentas seria suficiente: o Prisma ORM™ é amplamente empregado para operações CRUD com tipagem forte e integração nativa ao TypeScript, enquanto o Mongoose™ é tradicionalmente utilizado para modelagem detalhada, validação e manipulação avançada de dados em MongoDB™. No entanto, no contexto do SecretarIA, a combinação de ambas proporciona maior produtividade, segurança e controle sobre a integridade dos dados.

Prisma ORM™: Produtividade e Segurança no Acesso aos Dados

O Prisma ORM™ (Object-Relational Mapping) permite interagir com bancos de dados usando JavaScript ou TypeScript, abstraindo comandos diretos e reduzindo a incidência de erros. Sua integração nativa com TypeScript garante tipagem forte, proporcionando maior segurança em tempo de desenvolvimento (PRISMA, 2024).

Como funciona:

- O desenvolvedor define os modelos de dados em um arquivo de configuração (schema).
- O Prisma ORM™ gera funções para acessar e manipular esses dados no banco.
- A aplicação utiliza essas funções para operações como cadastro de usuários, eventos e senhas.

Vantagens:

- **Produtividade:** Reduz código repetitivo e acelera o desenvolvimento.
- **Segurança:** Minimiza riscos de erros de sintaxe e vulnerabilidades como injeção de código.
- **Tipagem forte:** Integração total com TypeScript.

No SecretarIA:

- Utilizado para operações principais de leitura e escrita, com consultas complexas e seguras.

Mongoose™: Modelagem e Validação Detalhada de Dados

O Mongoose™ é uma biblioteca para Node.js™ voltada à modelagem, validação e manipulação avançada de dados em MongoDB™. Permite definir esquemas rigorosos, garantindo que os dados armazenados estejam sempre no formato esperado (MONGOOSE, 2024).

Como funciona:

- O desenvolvedor define esquemas detalhados para cada tipo de dado.
- Antes de salvar ou atualizar, o Mongoose™ valida os dados conforme as regras estabelecidas.
- Oferece recursos avançados, como hooks, middlewares e relacionamentos.

Vantagens:

- **Validação automática:** Garante a integridade dos dados.
- **Modelagem clara:** Facilita a documentação e manutenção.
- **Recursos avançados:** Suporte a regras de negócio e manipulação granular.

No SecretarIA:

- Utilizado para validação rigorosa e modelagem detalhada, prevenindo inconsistências e facilitando a evolução do sistema.

Por que usar ambos?

Enquanto o Prisma ORM™ oferece produtividade, tipagem forte e integração eficiente com TypeScript, o Mongoose™ complementa com recursos avançados de modelagem e validação. Em sistemas menores ou menos exigentes, apenas um deles seria suficiente. No entanto, no SecretarIA, a combinação permite aproveitar o melhor dos dois mundos: rapidez no desenvolvimento, segurança na manipulação dos dados e controle rigoroso sobre a estrutura e integridade das informações.

2.4.7 Tailwind CSS™

O Tailwind CSS™ é um framework de utilitários para CSS que revolucionou a forma como a interface do SecretarIA foi construída (LABS, 2024). Diferente dos

frameworks tradicionais, o Tailwind CSS™ permite aplicar estilos diretamente nos elementos HTML/JSX usando classes utilitárias, como `bg-blue-500` (cor de fundo azul) ou `p-4` (padding de 1rem).

Como funciona na prática:

- Cada componente da interface recebe classes Tailwind CSS™ que definem cor, espaçamento, tipografia, bordas e responsividade.
- Não é necessário criar arquivos CSS separados para cada novo estilo, pois as classes utilitárias já cobrem praticamente todas as necessidades visuais.
- O sistema de responsividade do Tailwind CSS™ permite adaptar a interface para diferentes tamanhos de tela usando classes como `md:w-1/2` (largura de metade da tela em telas médias para cima).

Exemplo prático no SecretarIA:

```
<button className="bg-purple-600 hover:bg-purple-700 text-white font-bo
  Criar Evento
</button>
```

Esse botão, com poucas classes, já possui cor, efeito ao passar o mouse, tipografia destacada, espaçamento interno e bordas arredondadas — tudo sem CSS adicional.

Vantagens para o SecretarIA:

- **Agilidade no desenvolvimento:** A equipe pode prototipar e ajustar a interface rapidamente, sem depender de longos ciclos de edição de CSS.
- **Consistência visual:** Como todos usam as mesmas classes, o visual do sistema permanece padronizado e profissional.
- **Facilidade de manutenção:** Alterações no design são feitas diretamente nos componentes, facilitando correções e melhorias.
- **Responsividade nativa:** A interface se adapta automaticamente a celulares, tablets e desktops, garantindo boa experiência para todos os usuários.

No SecretarIA:

- Todas as telas (login, agenda, cadastro, notificações) foram construídas com Tailwind CSS™, garantindo visual moderno e usabilidade.
- O design limpo e intuitivo foi possível graças à flexibilidade das classes utilitárias, que permitiram criar layouts personalizados sem complexidade.

2.5 Ferramentas de Apoio ao Desenvolvimento

O desenvolvimento do SecretarIA contou com o apoio de diversas ferramentas que facilitaram a codificação, o gerenciamento de dados e o controle de versões, tornando o processo mais ágil, seguro e colaborativo.

Visual Studio Code™: Edição de Código Moderna e Produtiva

O Visual Studio Code™ (VS Code) foi o editor de código principal utilizado durante o desenvolvimento. Ele oferece recursos como destaque de sintaxe, autocompletar inteligente, depurador integrado e uma vasta biblioteca de extensões.

Como foi usado:

- Codificação dos componentes do frontend (React™, Next.js™) e backend (Node.js™).
- Uso de extensões para linting, formatação automática e integração com Git™.
- Depuração de código e execução de testes diretamente no editor.

Vantagens:

- Aumenta a produtividade com recursos inteligentes.
- Facilita a identificação e correção de erros.
- Permite personalização conforme as necessidades do projeto.

DBeaver™: Administração Visual do Banco de Dados

O DBeaver™ é um cliente gráfico para bancos de dados, utilizado para visualizar, editar e administrar as coleções do MongoDB™ durante o desenvolvimento.

Como foi usado:

- Visualização e edição de dados armazenados (eventos, usuários, senhas).

- Execução de consultas para depuração e validação dos dados.
- Verificação da estrutura das coleções e integridade das informações.

Vantagens:

- Interface amigável e intuitiva.
- Facilita a inspeção e modificação de dados sem necessidade de comandos complexos.
- Auxilia na detecção rápida de inconsistências durante os testes.

Git™: Controle de Versão e Colaboração

O Git™ foi utilizado para versionar todo o código-fonte do SecretarIA, permitindo o rastreamento de alterações, colaboração entre desenvolvedores e recuperação de versões anteriores do projeto.

Como foi usado:

- Criação de repositório para armazenar o código e o histórico de alterações.
- Ramificações (branches) para desenvolvimento de novas funcionalidades sem afetar a versão principal.
- Integração com plataformas como GitHub™ para colaboração, revisão de código e gerenciamento de issues.

Vantagens:

- Garante a segurança do código e a possibilidade de reverter mudanças indesejadas.
- Facilita o trabalho em equipe, mesmo remotamente.
- Permite acompanhamento detalhado da evolução do projeto.

Resumo: O uso integrado dessas ferramentas proporcionou um fluxo de trabalho eficiente, seguro e colaborativo, contribuindo diretamente para a qualidade e a organização do desenvolvimento do SecretarIA.

2.6 Segurança e Privacidade

A segurança foi tratada como prioridade em todas as etapas do desenvolvimento do SecretarIA. Para proteger as informações dos usuários, foram adotadas as seguintes práticas:

- **Hashing de senhas:** As senhas dos usuários são protegidas com hashing usando o algoritmo bcrypt, tornando impossível recuperar a senha original a partir do dado armazenado.
- **Criptografia simétrica (AES):** Dados sensíveis, como senhas de aplicativos, tokens de autenticação e informações pessoais, são protegidos com criptografia simétrica utilizando o algoritmo AES. *Criptografia simétrica* é um método em que a mesma chave é usada tanto para cifrar quanto para decifrar as informações.
- **Autenticação robusta:** O acesso ao sistema é controlado via NextAuth™, garantindo que apenas usuários autorizados possam visualizar ou modificar dados.

No contexto do SecretarIA, **dados sensíveis** incluem informações que, se expostas, podem comprometer a privacidade ou a segurança do usuário, como senhas de acesso, tokens de autenticação, endereços de e-mail e registros de eventos pessoais.

Além dessas práticas, foram seguidas as recomendações do OWASP Top 10 (OWASP Foundation, 2021), para mitigar vulnerabilidades comuns e o sistema foi projetado em conformidade com a Lei Geral de Proteção de dados (LGPD) (BRASIL, 2018), garantindo transparência e proteção dos dados dos usuários.

2.7 Estratégias de Testes

Garantir a qualidade e a confiabilidade do SecretarIA foi uma preocupação central durante o desenvolvimento. Para isso, adotou-se uma abordagem estruturada de testes, utilizando ferramentas modernas e práticas recomendadas pelo mercado.

Por que testar?

Os testes automatizados são fundamentais para:

- Detectar erros e falhas rapidamente, antes que cheguem ao usuário final.

- Garantir que novas funcionalidades não quebrem partes já implementadas (regressão).
- Facilitar a manutenção e evolução do sistema, permitindo mudanças seguras no código.

Como testamos?

Foram realizados dois tipos principais de testes:

- **Testes unitários:** Verificam se funções e componentes individuais funcionam corretamente de forma isolada.
- **Testes de integração:** Avaliam se diferentes partes do sistema funcionam bem juntas, simulando fluxos reais de uso.

Esses testes foram escritos e executados ao longo de todo o desenvolvimento, acompanhando cada nova funcionalidade implementada.

Por que Jest™ e Testing Library™?

A escolha do Jest™ e da Testing Library™ se deu por sua integração nativa com o ecossistema React™/Next.js™, ampla adoção na comunidade e documentação abrangente, o que facilita a resolução de dúvidas e a evolução dos testes (PLATFORMS, 2024; LIBRARY, 2024).

- **Jest™:** Framework de testes robusto, rápido e fácil de configurar, amplamente utilizado em projetos JavaScript/TypeScript. Permite criar testes unitários e de integração de forma eficiente.
- **Testing Library™:** Focada em testar interfaces de usuário (React™), simula interações reais do usuário, garantindo que a aplicação funcione como esperado do ponto de vista de quem a utiliza.

No SecretarIA:

- Foram testados componentes críticos como o calendário, o gerenciador de senhas e o sistema de notificações.

- Simularam-se cenários reais, como criação de eventos, login e recebimento de notificações, validando o comportamento esperado.
- O feedback dos usuários foi utilizado para identificar casos de uso importantes e aprimorar os testes.

2.8 Metodologia de Desenvolvimento

O desenvolvimento do SecretarIA seguiu práticas ágeis, caracterizadas por ciclos curtos de planejamento, execução e revisão. Metodologias ágeis, como Scrum e Kanban, priorizam a adaptação rápida às mudanças e a entrega contínua de valor ao usuário (SCHWABER; BEEDLE, 2002; ANDERSON, 2010). Essas abordagens promovem colaboração, transparência e flexibilidade, permitindo que o time responda prontamente a novos requisitos e feedbacks.

No projeto, foram realizados planejamentos de sprints, revisões periódicas e integração contínua. As decisões de arquitetura e implementação basearam-se em revisões bibliográficas, análise de requisitos e testes iterativos, possibilitando ajustes rápidos conforme o feedback obtido durante o processo. Essa abordagem garantiu maior alinhamento com as necessidades dos usuários e maior qualidade no produto final.

3 METODOLOGIA

Este capítulo detalha a abordagem metodológica adotada no desenvolvimento do SecretarIA, descrevendo as fases do projeto, as ferramentas e tecnologias utilizadas, os métodos de avaliação do protótipo e as considerações éticas envolvidas.

3.1 Abordagem Metodológica

O desenvolvimento do SecretarIA foi guiado por uma abordagem iterativa e incremental, inspirada nos princípios das metodologias ágeis, porém adaptada à realidade de um projeto individual de TCC (ver Seção 3.2). Essa escolha metodológica se mostrou especialmente adequada diante das incertezas técnicas inerentes à integração de tecnologias emergentes, como modelos de linguagem grandes (LLMs)(ZHANG et al., 2024) e frameworks de orquestração de IA(KUMAR, 2024), além da necessidade de obter resultados parciais rapidamente para validação contínua.

O processo foi organizado em ciclos curtos de desenvolvimento, nos quais funcionalidades eram planejadas, implementadas, testadas e integradas progressivamente ao sistema. A cada iteração, o escopo era revisado e ajustado conforme os aprendizados e desafios encontrados, promovendo flexibilidade e adaptação. O foco esteve sempre na entrega de valor incremental, permitindo que o protótipo evoluísse de um núcleo funcional básico para um sistema mais completo e robusto.

Além disso, mesmo sem um cliente externo formal, o próprio desenvolvedor desempenhou o papel de avaliador contínuo, realizando testes manuais e identificando oportunidades de melhoria a cada ciclo. Essa dinâmica favoreceu a identificação e resolução ágil de problemas, bem como a adaptação do projeto às necessidades reais que surgiram durante sua execução.

3.2 Adaptações Metodológicas para Projeto Individual de TCC

Embora inspirado em métodos ágeis tradicionais (como Scrum e Kanban), o desenvolvimento do SecretarIA exigiu adaptações importantes devido ao contexto de um projeto individual e acadêmico. As principais adaptações foram:

- **Planejamento flexível e simplificado:** Não houve divisão formal em sprints ou reuniões de equipe. O planejamento foi feito em ciclos curtos, com revisões frequentes de prioridades conforme o avanço do trabalho e surgimento de desafios técnicos.
- **Validação contínua com autoavaliação:** Em vez de revisões por equipe, o progresso foi avaliado por meio de testes manuais, feedback do orientador e autoavaliação crítica dos resultados parciais.
- **Documentação adaptada:** A documentação foi priorizada para facilitar a redação do TCC e o registro de decisões técnicas, substituindo artefatos tradicionais de metodologias ágeis.
- **Gestão de tempo individualizada:** O cronograma foi ajustado de acordo com as demandas acadêmicas, disponibilidade pessoal e prazos institucionais, sem dependências externas.
- **Foco em entregas parciais e prototipagem rápida:** O desenvolvimento priorizou a obtenção de versões funcionais mínimas (MVPs) para cada módulo, permitindo validação e ajustes antes da implementação completa.

Essas adaptações garantiram flexibilidade, autonomia e foco nos objetivos essenciais do TCC, tornando a abordagem metodológica mais adequada à realidade de um projeto individual.

3.3 Fases do Projeto

O desenvolvimento do SecretarIA foi dividido em etapas bem definidas, cada uma com objetivos, desafios e decisões próprias. A seguir, apresento cada fase, detalhando o que foi realizado, os resultados obtidos e as justificativas para as escolhas feitas.

3.3.1 Pesquisa e Definição

Nesta fase inicial, realizei uma revisão bibliográfica sobre assistentes pessoais inteligentes, modelos de linguagem natural (LLMs), frameworks de IA generativa, segu-

rança da informação e experiência do usuário. O objetivo foi embasar tecnicamente as decisões do projeto e identificar boas práticas de mercado.

Resultados e escolhas:

- Definição do escopo do SecretarIA, priorizando funcionalidades essenciais como agenda, gerenciamento de senhas, notificações e assistente inteligente.
- Levantamento dos requisitos funcionais (o que o sistema deve fazer) e não funcionais (segurança, desempenho, usabilidade).
- Escolha da arquitetura de três camadas, visando modularidade e facilidade de manutenção.
- Seleção de tecnologias modernas (Next.js™, React™, MongoDB™, OpenAI™, etc.) por sua robustez e integração com IA.

3.3.2 Configuração Inicial

Com base nas definições anteriores, preparei o ambiente de desenvolvimento para garantir produtividade e organização desde o início.

Resultados e escolhas:

- Instalação e configuração do Node.js™ e do Visual Studio Code™ como base para o desenvolvimento.
- Criação do repositório Git™ para versionamento, permitindo controle de alterações e colaboração.
- Estruturação inicial do projeto com Next.js™, facilitando a integração entre frontend e backend.
- Integração do Tailwind CSS™ para estilização ágil e consistente.
- Configuração do NextAuth™ para autenticação segura desde o início do projeto.

3.3.3 Desenvolvimento dos Módulos Principais

O desenvolvimento seguiu uma abordagem incremental, priorizando a entrega de funcionalidades essenciais em ciclos curtos e iterativos.

Resultados e escolhas:

- Implementação do sistema de autenticação, garantindo segurança desde a primeira versão funcional.
- Desenvolvimento do módulo de agenda, permitindo cadastro, edição e visualização de compromissos.
- Criação do gerenciador de senhas, com armazenamento seguro e criptografado.
- Integração do chat inteligente, utilizando LangChain™ e OpenAI™ para interação natural com o usuário.
- Cada módulo foi testado e integrado gradualmente, permitindo ajustes rápidos conforme feedback e testes.

3.3.4 Refinamento e Testes

Após a implementação dos módulos principais, o foco foi aprimorar a qualidade, segurança e usabilidade do sistema.

Resultados e escolhas:

- Refatoração do código para melhorar legibilidade, desempenho e manutenibilidade.
- Aprimoramento da interface com Tailwind CSS™, tornando o sistema mais intuitivo e agradável.
- Implementação de testes unitários e de integração com Jest™ e Testing Library™, garantindo robustez das funcionalidades.
- Revisão das medidas de segurança, seguindo recomendações do OWASP Top 10.
- Realização de testes de usabilidade e coleta de feedback de usuários para ajustes finais.

3.3.5 Documentação e Redação Final

Por fim, dediquei-se à documentação detalhada do projeto e à preparação do trabalho acadêmico.

Resultados e escolhas:

- Elaboração dos capítulos do TCC, descrevendo todas as fases, decisões e resultados.
- Produção de diagramas, fluxogramas e capturas de tela ilustrando a arquitetura e o funcionamento do sistema.
- Revisão e formatação do texto conforme as normas da instituição, garantindo clareza e organização.

3.4 Ferramentas e Tecnologias

A escolha das ferramentas e tecnologias do SecretarIA foi guiada por critérios como popularidade, robustez, qualidade da documentação, aderência aos requisitos do projeto e experiência prévia do desenvolvedor. A seguir, detalho cada uma das principais escolhas, justificando o porquê de sua adoção e como foram aplicadas no sistema.

3.4.1 JavaScript™/TypeScript™

Essas linguagens foram escolhidas por permitirem o desenvolvimento full-stack, ou seja, tanto do frontend quanto do backend, aproveitando o vasto ecossistema de pacotes NPM(ABDALKAREEM et al., 2017). O TypeScript™ foi adotado para trazer tipagem estática, aumentando a segurança e reduzindo erros em tempo de desenvolvimento.

3.4.2 Next.js™ e React™

O Next.js™ é um framework moderno baseado em React™, que permite criar aplicações web escaláveis com renderização híbrida (SSR, SSG, CSR). Foi escolhido por sua integração facilitada entre frontend e backend(RAUCH; CONTRIBUTORS,

2020), além de recursos como rotas de API e otimização de desempenho. O React™ possibilitou a criação de interfaces modulares e reutilizáveis, acelerando o desenvolvimento e facilitando a manutenção.

3.4.3 Tailwind CSS™

O Tailwind CSS™ foi adotado para estilização da interface, pois permite aplicar estilos diretamente nas classes dos componentes(WATHAN et al., 2020). Isso agilizou a criação de layouts responsivos e consistentes, reduzindo a necessidade de arquivos CSS separados e facilitando ajustes visuais rápidos.

3.4.4 MongoDB Atlas™

Optou-se pelo MongoDB Atlas™ como banco de dados NoSQL orientado a documentos devido à sua flexibilidade de esquema, facilidade de integração com aplicações JavaScript™ e escalabilidade(BANKER et al., 2016). O uso do serviço em nuvem simplificou a gestão, backup e monitoramento do banco de dados.

3.4.5 Prisma ORM™/Mongoose™

O Prisma ORM™ foi utilizado para facilitar o acesso, manipulação e consulta dos dados no MongoDB™, trazendo produtividade e segurança ao desenvolvimento(PRISMA, 2022). O Mongoose™ foi empregado para modelagem e validação dos dados, garantindo que as informações armazenadas estejam sempre no formato correto(KARPOV, 2018).

3.4.6 NextAuth.js™

A autenticação dos usuários foi implementada com NextAuth.js™, devido à sua robustez, suporte a múltiplos provedores (Google™, e-mail, etc.) e facilidade de integração com o Next.js™(CONTRIBUTORS, 2023a). Isso garantiu segurança e flexibilidade no controle de acesso ao sistema.

3.4.7 LangChain™ e OpenAI™ (GPT-3.5/4)

Para a camada de inteligência artificial, foi utilizado o framework LangChain™ (CONTRIBUTORS, 2023b) em conjunto com os modelos de linguagem da OpenAI™(OPENAI, 2023). O LangChain™ facilitou a orquestração de prompts, gerenciamento de contexto e integração com LLMs, enquanto a OpenAI™ forneceu respostas inteligentes e contextualizadas para o assistente virtual.

3.4.8 Nodemailer™

O envio de e-mails automáticos, como lembretes de eventos e confirmações de cadastro, foi realizado com o Nodemailer™. A escolha se deu pela simplicidade de uso, flexibilidade e ampla compatibilidade com diferentes serviços de e-mail(REINMAN, 2023).

3.4.9 Visual Studio Code™, Git™ e GitHub™

O Visual Studio Code™ foi o editor padrão, oferecendo recursos avançados para produtividade e depuração(MICROSOFT, 2023). O Git™ foi utilizado para controle de versão, permitindo rastreamento de alterações e trabalho colaborativo(CHACON; STRAUB, 2014). O GitHub™ serviu como plataforma para hospedagem do repositório, revisão de código e gerenciamento de *issues*(GITHUB, 2023).

Resumo: Cada tecnologia foi escolhida visando maximizar produtividade, segurança, escalabilidade e facilidade de manutenção, garantindo que o SecretarIA atendesse aos objetivos propostos com qualidade e eficiência.

3.5 Avaliação do Protótipo

É importante ressaltar, logo de início, que não foi possível realizar uma avaliação rigorosa e robusta do protótipo do SecretarIA. Devido à natureza individual do projeto, limitações de tempo e recursos, a avaliação realizada não envolveu testes formais com usuários finais nem métodos quantitativos estruturados.

Dessa forma, a análise do sistema concentrou-se em métodos qualitativos e na verificação interna das funcionalidades implementadas, com foco nos seguintes aspectos:

- **Análise Funcional:** Foram realizados testes manuais para conferir se todos os requisitos funcionais estavam presentes, incluindo fluxos de autenticação, cadastro, gerenciamento de eventos, senhas e interação via chat.
- **Avaliação Heurística de Usabilidade:** Utilizaram-se as heurísticas de Nielsen para identificar possíveis problemas de usabilidade, clareza das informações, feedback ao usuário e tratamento de erros.
- **Análise da Interação Conversacional:** Foram feitos testes específicos no chat, avaliando a precisão da compreensão das intenções do usuário, extração de informações relevantes e coerência das respostas geradas pela IA.
- **Revisão das Medidas de Segurança:** Realizou-se uma análise crítica das práticas de autenticação, criptografia e controle de acesso, baseando-se nas recomendações do OWASP Top 10 e nos requisitos não funcionais definidos.

Reconheço que a ausência de testes quantitativos e de validação com usuários reais limita o rigor e a abrangência da avaliação. No entanto, a análise crítica permitiu identificar pontos fortes do sistema, bem como oportunidades de melhoria, que são discutidos nos capítulos de resultados e discussão.

4 DESENVOLVIMENTO

O desenvolvimento do SecretarIA foi um processo que envolveu desde a concepção da ideia até a superação de desafios técnicos e de design, sempre com o objetivo de entregar uma solução robusta, segura e realmente útil ao usuário final. Nesta seção, detalho o percurso, as decisões e os aprendizados que marcaram a construção do sistema.

4.1 Requisitos do Sistema

Os requisitos do SecretarIA foram definidos a partir da análise das necessidades dos usuários e dos objetivos do sistema. Eles estão classificados em requisitos funcionais (RF), que descrevem as funcionalidades essenciais, e requisitos não funcionais (RNF), que tratam das características de qualidade e restrições técnicas.

4.1.1 Requisitos Funcionais

A tabela a seguir lista o código, nome e classificação dos requisitos funcionais. Eles foram classificados em essencial (E), importante (I) e desejável (D). Logo após, é apresentada a descrição detalhada de cada requisito.

Tabela 1 – Requisitos Funcionais

Código	Descrição	Classificação
RF01	Cadastro de Usuário	Essencial
RF02	Login de Usuário	Essencial
RF03	Recuperação de Senha	Importante
RF04	Visualizar Calendário	Essencial
RF05	Adicionar Evento Manualmente	Essencial
RF06	Editar Evento	Importante
RF07	Excluir Evento	Importante
RF08	Listar Credenciais de Senha	Essencial
RF09	Adicionar/Editar/Excluir Credencial de Senha	Essencial
RF10	Enviar Mensagem para IA (Chat)	Essencial
RF11	Receber Resposta da IA (Chat)	Essencial
RF12	Consultar Agenda via Chat	Essencial
RF13	Adicionar, Editar e Excluir Evento via Chat	Essencial

A seguir, cada requisito funcional é detalhado:

- **RF01: Cadastro de Usuário** – O sistema deve permitir que novos usuários realizem seu cadastro, informando e validando dados essenciais.
- **RF02: Login de Usuário** – O sistema deve permitir autenticação via e-mail/senha (com hash seguro) ou Google OAuth.
- **RF03: Recuperação de Senha** – O usuário deve poder recuperar sua senha por e-mail de forma segura.
- **RF04: Visualizar Calendário** – O usuário deve visualizar seus eventos em diferentes formatos, como mensal, semanal ou agenda.
- **RF05: Adicionar Evento Manualmente** – O usuário pode criar novos eventos, informando data, hora, título e descrição, via interface gráfica.
- **RF06: Editar Evento** – O usuário pode editar eventos já cadastrados.
- **RF07: Excluir Evento** – O usuário pode remover eventos já cadastrados.
- **RF08: Listar Credenciais de Senha** – O sistema exibe uma lista das credenciais cadastradas pelo usuário, com senhas mascaradas por padrão.
- **RF09: Adicionar/Editar/Excluir Credencial de Senha** – O usuário pode gerenciar credenciais, sempre com criptografia.
- **RF10: Enviar Mensagem para IA (Chat)** – O usuário pode enviar mensagens para a IA via interface de chat.
- **RF11: Receber Resposta da IA (Chat)** – O usuário recebe respostas da IA na interface de chat.
- **RF12: Consultar Agenda via Chat** – O usuário pode consultar sua agenda por meio de comandos em linguagem natural.
- **RF13: Adicionar, Editar e Excluir Evento via Chat** – O usuário pode criar, editar e excluir eventos usando comandos no chat inteligente.

4.1.2 Requisitos Não Funcionais

Os requisitos não funcionais estabelecem restrições técnicas e padrões de qualidade para o sistema. A tabela a seguir apresenta os principais RNFs do SecretarIA:

Tabela 2 – Requisitos Não Funcionais

Requisito	Nome	Classificação
RNF01	Linguagem e Framework	Essencial
RNF02	Banco de Dados	Essencial
RNF03	Segurança	Essencial
RNF04	Hospedagem	Importante
RNF05	Responsividade	Essencial
RNF06	Performance	Importante
RNF07	Privacidade	Essencial
RNF08	Testabilidade	Importante

A seguir, cada requisito não funcional é detalhado:

- **RNF01: Linguagem e Framework** – O sistema deve ser desenvolvido em Next.js (React) para garantir modernidade e escalabilidade.
- **RNF02: Banco de Dados** – Persistência dos dados deve ser feita no MongoDB Atlas.
- **RNF03: Segurança** – Senhas e dados sensíveis devem ser criptografados.
- **RNF04: Hospedagem** – O sistema deve ser hospedado em ambiente seguro, com deploy contínuo.
- **RNF05: Responsividade** – O frontend deve se adaptar a diferentes tamanhos de tela.
- **RNF06: Performance** – O sistema deve apresentar respostas rápidas para garantir boa experiência.
- **RNF07: Privacidade** – O sistema deve estar em conformidade com a LGPD.
- **RNF08: Testabilidade** – O código deve ser testável, com cobertura de testes automatizados.

4.1.3 Visão Geral do Sistema

O SecretarIA foi idealizado como um assistente pessoal inteligente, capaz de centralizar funcionalidades essenciais para a organização da rotina e a segurança digital do usuário. Diferentemente de soluções fragmentadas, o SecretarIA integra agenda, gerenciador de senhas e um chat inteligente, tudo em uma única interface moderna, responsiva e de fácil uso. Desde o início, a preocupação com privacidade, segurança e acessibilidade foi central, guiando escolhas tecnológicas e de experiência do usuário.

A arquitetura proposta também facilita futuras integrações, como notificações push ou novos módulos de inteligência artificial, tornando o SecretarIA uma plataforma extensível.

4.1.4 Arquitetura e Estrutura do Projeto

A arquitetura do SecretarIA segue o padrão de três camadas (apresentação, lógica de negócio e persistência de dados), promovendo modularidade, escalabilidade e facilidade de manutenção. O padrão de três camadas foi escolhido para desacoplar responsabilidades e facilitar a realização de testes. Foram aplicados padrões de projeto como Service Layer e Repository para promover organização e reutilização de código.

O frontend, desenvolvido em Next.js e React, é responsável pela interface e experiência do usuário. O backend utiliza as API Routes do Next.js para processar regras de negócio, autenticação, integração com IA e comunicação com o banco de dados MongoDB Atlas, modelado via Mongoose.

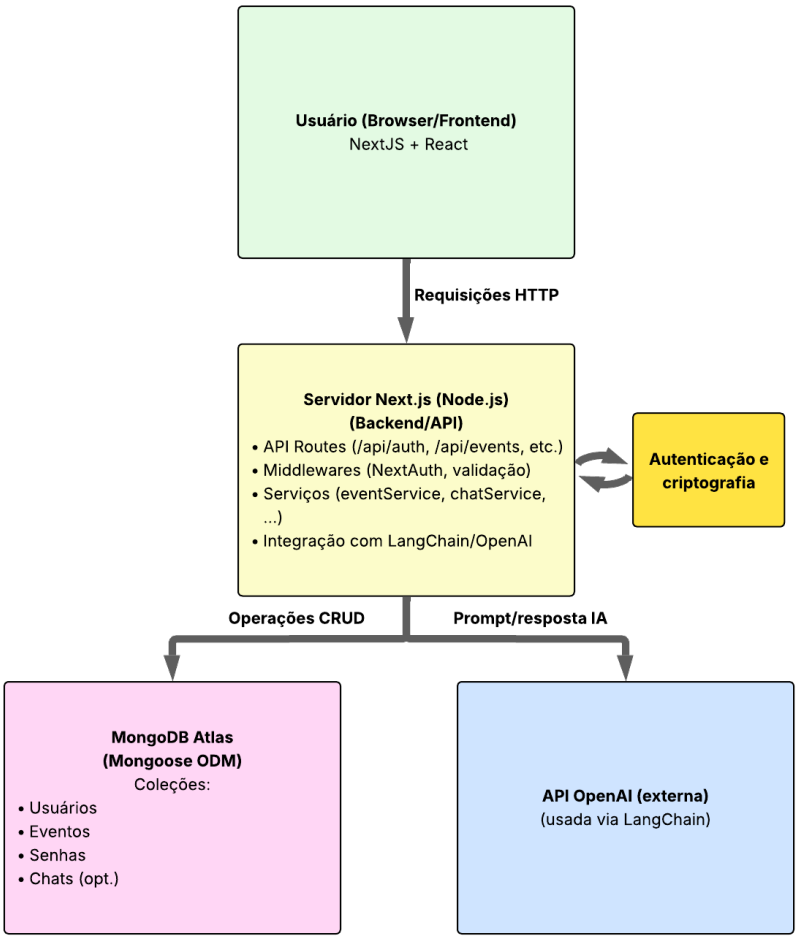


Figura 1 – Diagrama da arquitetura de três camadas do SecretarIA, destacando os fluxos de dados, autenticação e integração com serviços externos.

A estrutura de pastas foi organizada para facilitar a separação de responsabilidades, com diretórios distintos para componentes, páginas, rotas de API, serviços e utilitários. Isso permitiu um fluxo de trabalho mais limpo e colaborativo, mesmo em um projeto individual.

A escolha do Next.js se deu por sua capacidade de oferecer renderização híbrida (SSR/CSR), integração nativa entre frontend e backend e boa documentação, fatores essenciais para a produtividade e escalabilidade do projeto.

4.1.5 Detalhamento do Frontend

O frontend do SecretarIA foi pensado para ser intuitivo e objetivo. A componentização em React permitiu criar elementos reutilizáveis como *Navbar*, *Sidebar*, *CalendarView*, *PasswordList* e *ChatWindow*. O gerenciamento de estado global foi rea-

lizado com Context API, garantindo que informações como autenticação e preferências do usuário estivessem disponíveis em toda a aplicação.

O roteamento do Next.js, baseado em arquivos, facilitou a criação de páginas como `/login`, `/dashboard`, `/calendar`, `/passwords` e `/chat`. As rotas protegidas só são acessíveis após autenticação, reforçando a segurança.

A estilização foi feita com Tailwind CSS, proporcionando agilidade no desenvolvimento e um visual moderno. O design responsivo garante boa experiência tanto em desktops quanto em dispositivos móveis.

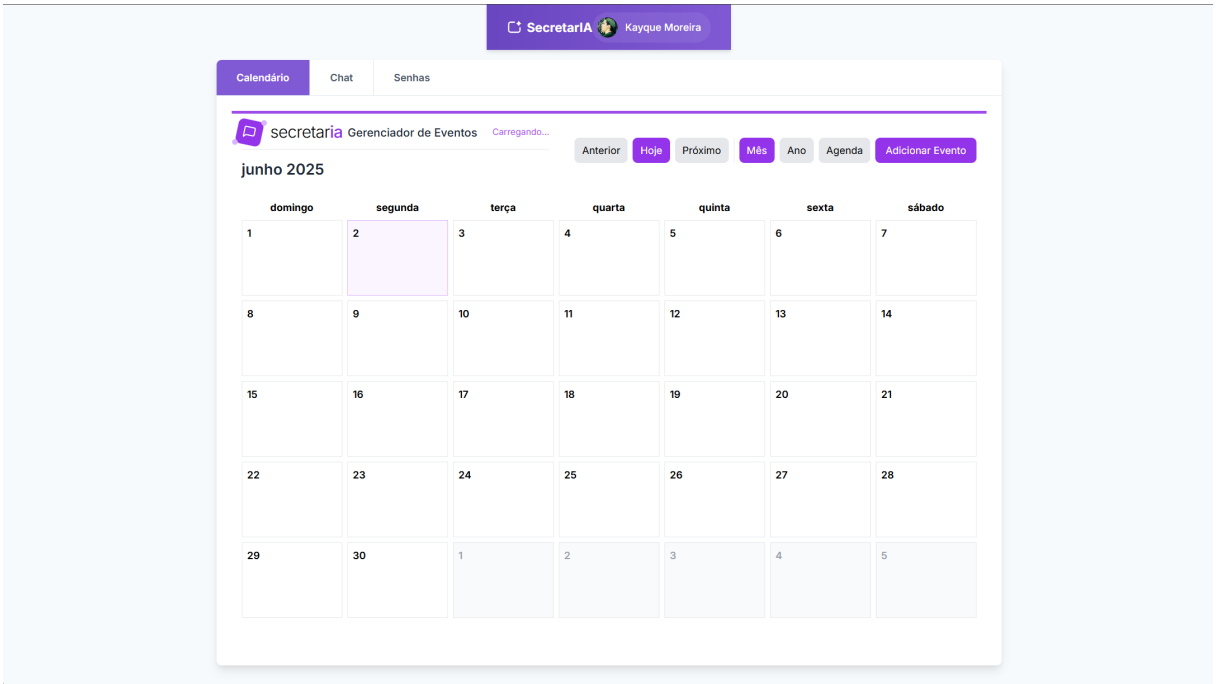


Figura 2 – Tela do dashboard do SecretarIA, destacando a integração dos módulos principais

Fluxo de uso: O usuário acessa o dashboard após autenticação, visualizando atalhos para os módulos principais. Ao clicar em "Calendário", pode ver, adicionar ou editar eventos. O acesso ao gerenciador de senhas e ao chat segue o mesmo padrão de navegação intuitiva.

4.1.6 Backend e Integração com Serviços

O backend do SecretarIA foi implementado nas API Routes do Next.js, organizadas em endpoints RESTful para autenticação, eventos, senhas e chat. Cada endpoint

conta com validação de entrada, autenticação via NextAuth e tratamento de erros, seguindo boas práticas de segurança (OWASP, 2021).

A lógica de negócio foi dividida em serviços especializados, como `eventService`, `passwordService` e `chatService`. Essa separação facilitou a manutenção, testes e futuras expansões.

A integração com IA foi realizada via LangChain e OpenAI, permitindo que o chat interpretasse comandos em linguagem natural, consultasse ou adicionasse eventos e respondesse de forma contextualizada. O backend é responsável por buscar o contexto relevante, montar o prompt e tratar a resposta da IA antes de retornar ao usuário.

Listing 4.1 – Exemplo simplificado de API Route para chat com IA

```
export default async function handler(req, res) {
  const { message } = req.body;
  const userSession = await getSession({ req });
  if (!userSession) return res.status(401).end();

  const context = await getUserEvents(userSession.user.id);
  const prompt = buildPrompt(message, context);

  const aiResponse = await langchain.callOpenAI(prompt);
  res.status(200).json({ response: aiResponse });
}
```

As rotas da API seguem o padrão REST e contam com validação de entrada utilizando a biblioteca Yup, garantindo integridade e segurança dos dados processados.

4.1.7 Persistência e Segurança dos Dados

O MongoDB Atlas foi escolhido como banco de dados pela flexibilidade e escalabilidade. O Mongoose facilitou a modelagem de entidades como Usuário, Evento, Senha e Histórico de Chat. Todas as operações de CRUD são protegidas por autenticação e restritas ao *userId* do usuário logado.

As senhas de login são armazenadas com hash bcrypt, enquanto as senhas salvas no gerenciador são criptografadas com AES-256 no backend. As chaves de

criptografia e segredos de autenticação são mantidos em variáveis de ambiente, nunca expostos ao frontend.

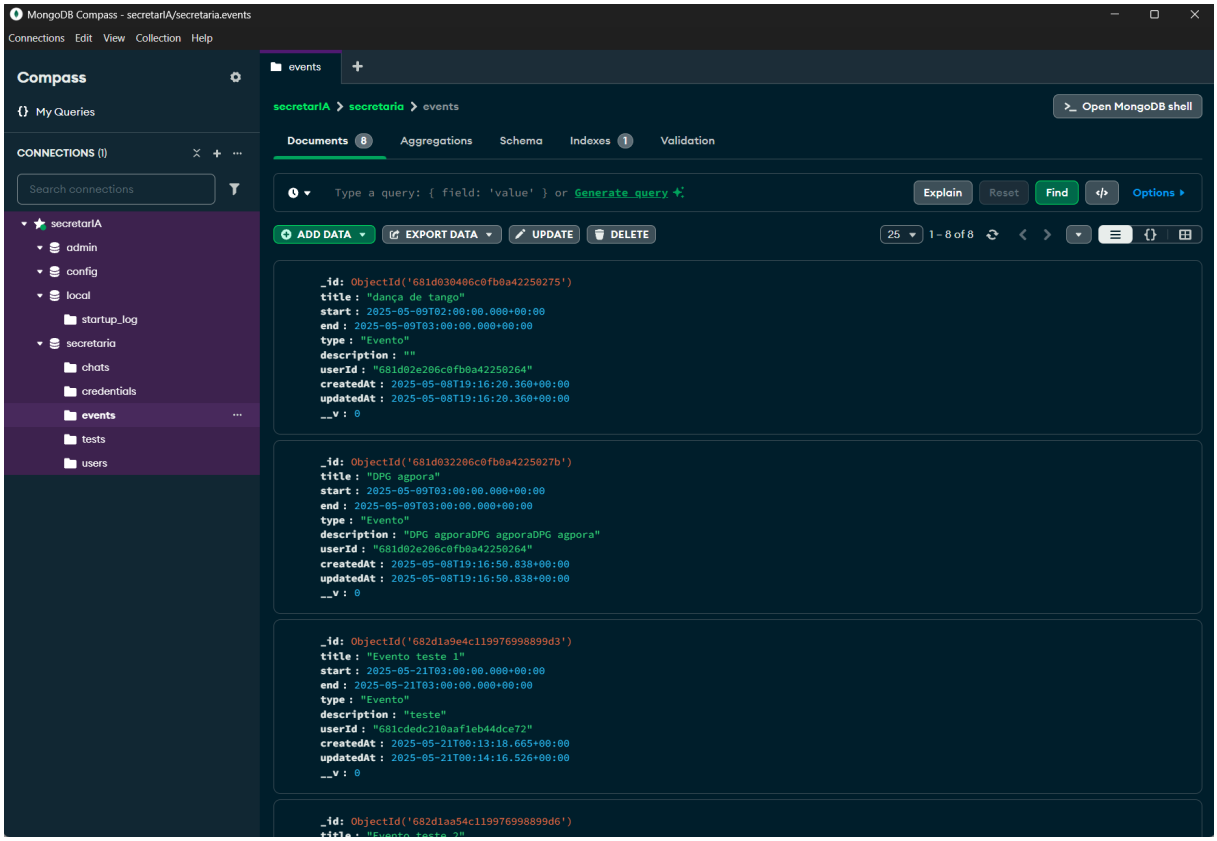


Figura 3 – Estrutura das coleções no MongoDB Atlas

O uso do tipo `ObjectId` como chave primária nas coleções facilita a escalabilidade do banco de dados. Além disso, foram implementados logs de acesso para auditoria e monitoramento de operações sensíveis.

A utilização de bcrypt e AES-256 foi motivada por serem padrões reconhecidos internacionalmente para proteção de credenciais (OWASP, 2021). O uso de variáveis de ambiente para segredos segue as melhores práticas de DevSecOps.

4.1.8 Funcionalidades-Chave do SecretarIA

4.1.9 Autenticação e Controle de Acesso

A autenticação é realizada via NextAuth, permitindo login por e-mail/senha (com hash seguro) ou Google OAuth. O controle de sessão é feito tanto no frontend

quanto no backend, protegendo rotas e garantindo que apenas usuários autorizados acessem suas informações.

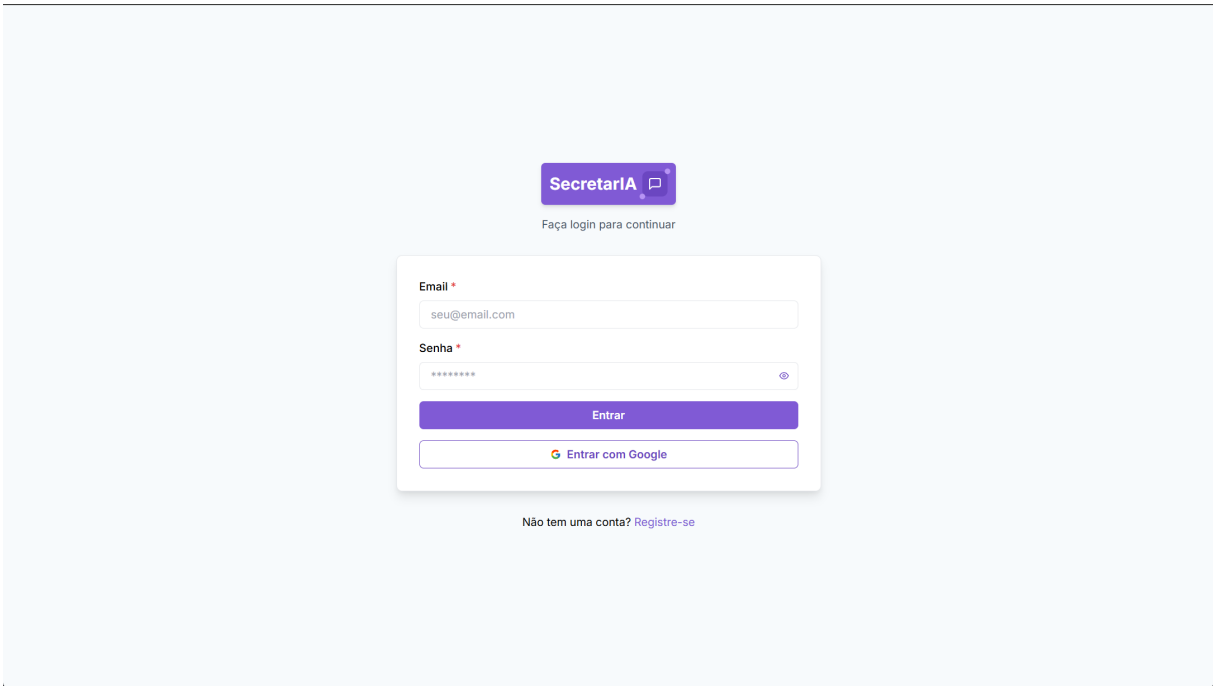


Figura 4 – Tela de login do SecretarIA

Fluxo de uso: O usuário acessa a tela de login, insere suas credenciais ou opta pelo login social Google. Após autenticação, é redirecionado ao dashboard.

4.1.10 Módulo Calendário

O calendário é o núcleo do SecretarIA. Utilizando uma biblioteca React especializada (ex: react-big-calendar), o usuário pode visualizar, adicionar, editar e excluir eventos em diferentes modos (mensal, anual, agenda). Modais intuitivos e feedback visual garantem uma experiência fluida.

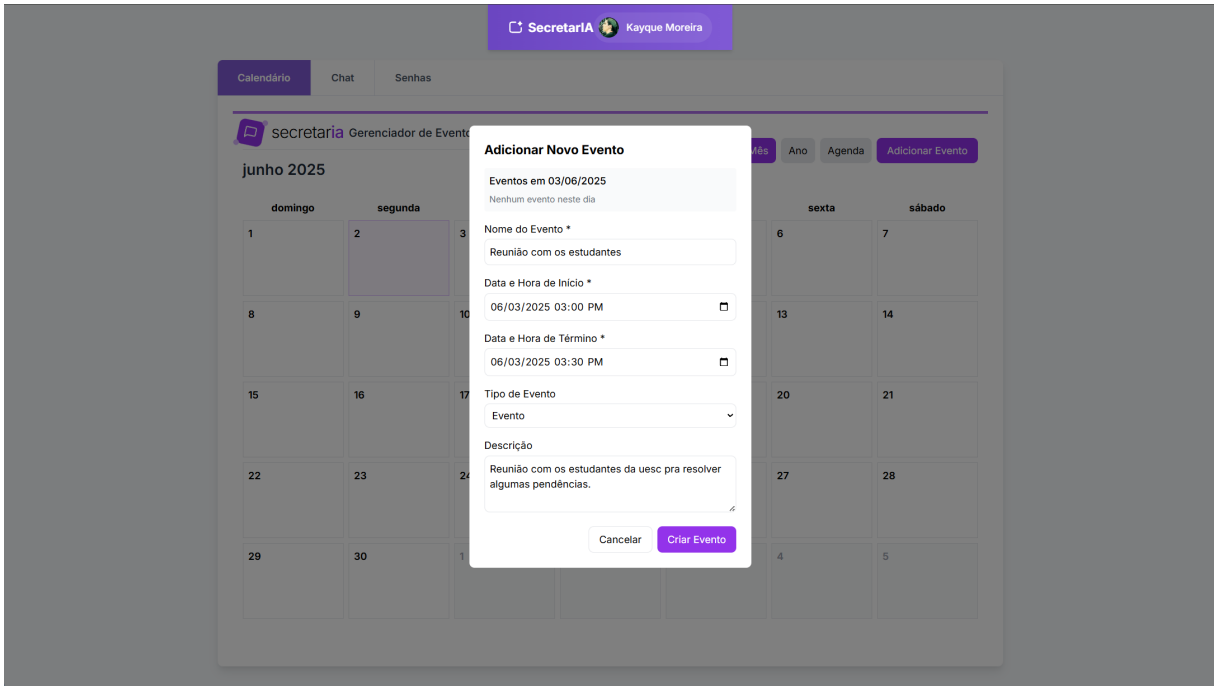


Figura 5 – Visualização do calendário e modal de evento

Fluxo de uso: Para adicionar um evento, o usuário clica em "+", preenche o formulário no modal e salva. O evento aparece imediatamente no calendário.

4.1.11 Gerenciador de Senhas

O Gerenciador de Senhas do SecretarIA foi projetado para armazenar credenciais de forma segura, prática e restrita à interface gráfica do sistema. Não é possível acessar, visualizar ou manipular senhas por meio do chat inteligente — essa separação foi adotada para reforçar a segurança e a privacidade dos dados sensíveis.

Como as senhas são armazenadas: Quando o usuário cadastra uma nova credencial (login e senha de um serviço), esses dados são enviados ao backend do SecretarIA. Antes de serem salvos no banco de dados MongoDB Atlas™, as senhas são criptografadas utilizando o algoritmo AES-256 (Advanced Encryption Standard). A chave de criptografia utilizada é armazenada de forma segura em variáveis de ambiente do servidor, nunca sendo exposta ao frontend ou ao usuário.

Como o usuário acessa suas senhas: Após realizar o login, o usuário pode acessar o módulo de Gerenciador de Senhas pelo menu principal da aplicação. As credenciais cadastradas são exibidas em uma tabela, com as senhas sempre mascara-

das por padrão (ou seja, ocultas). Para visualizar uma senha, o usuário deve clicar no botão de revelar, que solicita ao backend a descriptografia temporária daquela senha específica usando a chave AES-256. O valor é exibido apenas enquanto o usuário mantém o campo ativo, reforçando a proteção contra exposições acidentais.

Além disso, o usuário pode adicionar, editar ou excluir credenciais por meio de formulários próprios, sempre com validações para garantir a integridade e a segurança dos dados.

Fluxo típico de uso:

1. O usuário acessa o módulo após autenticação.
2. Visualiza a lista de credenciais, com senhas mascaradas.
3. Para ver uma senha, solicita a exibição temporária (com descriptografia controlada pelo backend e chave AES-256).
4. Pode adicionar, editar ou excluir credenciais usando formulários seguros.

Decisões de segurança:

- As senhas nunca são transmitidas ou exibidas via chat ou em texto claro.
- Toda operação sensível exige autenticação válida.
- A criptografia AES-256 garante proteção mesmo em caso de vazamento do banco de dados.
- A chave de criptografia é mantida exclusivamente no ambiente seguro do backend.
- O acesso às senhas é sempre individual e controlado, reforçando a privacidade do usuário.

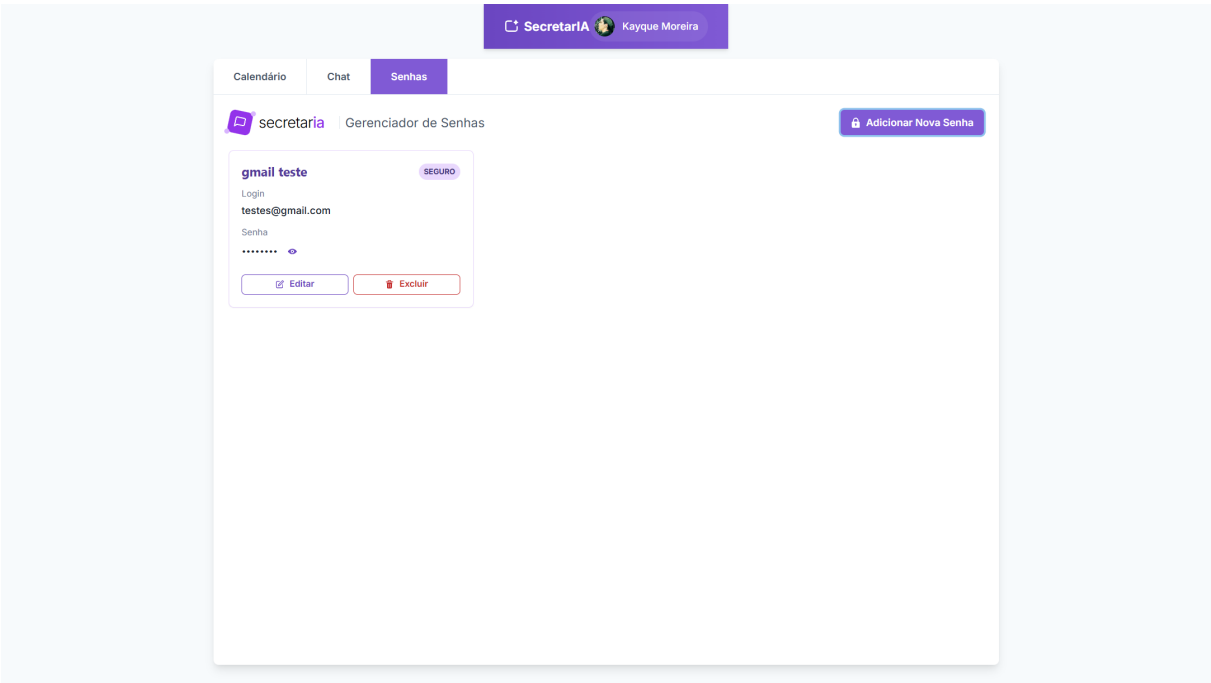


Figura 6 – Lista de senhas com campos mascarados

Resumo: O Gerenciador de Senhas oferece uma solução prática e robusta para armazenamento de credenciais, combinando criptografia forte (AES-256) e uma interface segura, sem integração com o chat inteligente para garantir a máxima proteção dos dados sensíveis.

4.1.12 Chat Inteligente

O chat inteligente do SecretarIA permite ao usuário interagir com o sistema em linguagem natural para realizar diversas tarefas, como consultar ou adicionar eventos, tirar dúvidas sobre funcionalidades ou pedir recomendações. No entanto, é importante destacar que o chat foi projetado para uso dentro do próprio sistema SecretarIA, sendo acessível apenas após o login do usuário. Não é possível utilizar o chat fora da plataforma, nem para comandos administrativos do sistema ou para operações que envolvam configurações avançadas de segurança.

Onde o chat pode ser usado:

- Consultar, adicionar, editar ou remover eventos do calendário pessoal.
- Solicitar lembretes e notificações de compromissos.

- Tirar dúvidas sobre o funcionamento do sistema e receber orientações básicas.
- Pedir recomendações sobre organização de tarefas e uso de funcionalidades.

Onde o chat não pode ser usado:

- Alterar configurações administrativas do sistema ou de outros usuários.
- Executar comandos que envolvam dados sensíveis de terceiros.
- Realizar operações fora do escopo do SecretarIA, como acessar outros sistemas ou bancos de dados externos.

Como funciona o chat inteligente:

Quando o usuário envia uma mensagem, o backend do SecretarIA executa três etapas principais:

1. **Busca do contexto:** O sistema identifica quem é o usuário, quais eventos já existem no calendário dele e qual foi o histórico recente de interações no chat. O contexto pode ser:
   - *Contexto de agenda:* Lista de compromissos, horários livres, eventos futuros e passados.
   - *Contexto de interação:* Mensagens anteriores, dúvidas já respondidas, preferências do usuário.
   - *Contexto de uso:* Ações recentes, como criação ou edição de eventos.
2. **Montagem do prompt:** O sistema organiza as informações do contexto e a mensagem do usuário em um texto estruturado (prompt), que é enviado ao modelo de IA. Isso garante que a resposta leve em conta a situação atual do usuário.
3. **Geração de resposta personalizada:** A IA analisa o prompt e gera uma resposta adaptada à necessidade do usuário, levando em conta tanto o pedido quanto o contexto. Uma resposta é considerada personalizada quando faz referência direta aos dados do usuário ou à situação específica dele.

Exemplo 1 – Agendamento de evento:

- *Usuário:* “Marque um evento para o dia 06/06 às 18 horas com título ir ao cinema ver Sinners.”
- *Processo:* O sistema verifica se já existe um evento nesse horário, monta o prompt com essa informação e envia para a IA.
- *Resposta da IA:* “Evento ‘ir ao cinema ver Sinners’ marcado com sucesso para o dia 06/06 às 18 horas! Se precisar de mais alguma coisa, é só me chamar.”

Exemplo 2 – Consulta de compromissos:

- *Usuário:* “Quais são meus compromissos amanhã?”
- *Processo:* O sistema busca todos os eventos do usuário para o dia seguinte, monta o prompt e envia para a IA.
- *Resposta da IA:* “Você tem dois compromissos amanhã: reunião de equipe às 10h e consulta médica às 16h.”

O chat inteligente, portanto, atua como um assistente que entende o contexto do usuário e responde de forma relevante, facilitando a organização e o uso eficiente das funcionalidades do SecretarIA.

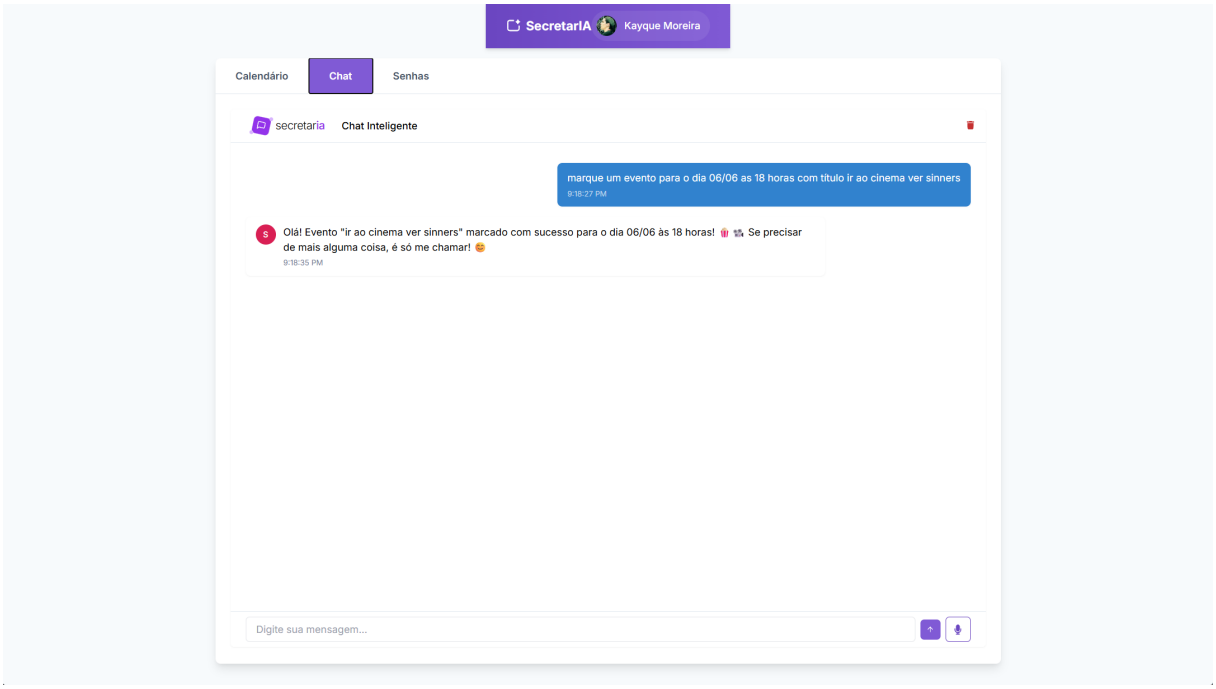


Figura 7 – Interação do usuário com o chat inteligente

4.1.13 Notificações por E-mail

O envio de notificações por e-mail, implementado via Nodemailer™, mantém o usuário informado sobre eventos importantes, como confirmações de agendamento, lembretes e alterações em compromissos. Essa funcionalidade reforça o compromisso do sistema com a comunicação eficiente e a experiência do usuário.

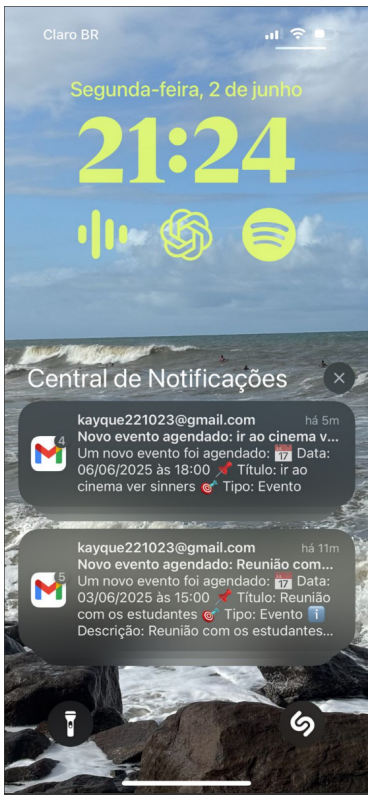


Figura 8 – Exemplo de notificação de e-mail enviado pelo sistema

Vale destacar que, ao receber um e-mail do SecretarIA, o usuário pode ser notificado automaticamente tanto em dispositivos Android quanto em iOS, desde que utilize aplicativos de e-mail (como Gmail™ ou Mail) com notificações ativas. Dessa forma, os avisos chegam rapidamente ao smartphone, funcionando como notificações push, sem necessidade de desenvolvimento adicional para aplicativos nativos.

Funcionalidades similares de notificação instantânea via WhatsApp™ não foram implementadas devido às tarifas cobradas pela Meta™ para envio de mensagens automatizadas, o que inviabilizaria o uso gratuito ou de baixo custo para o usuário final. Caso o SecretarIA evolua para uma versão comercial ou institucional, a integração com outros canais de comunicação poderá ser reavaliada.

4.1.14 Segurança, Privacidade e Boas Práticas

O SecretarIA foi desenvolvido seguindo as recomendações do OWASP Top 10 (OWASP Foundation, 2021), protegendo contra os principais riscos de segurança em aplicações web. A seguir, explico quem é a OWASP e detalho cada uma das proteções implementadas:

Quem é a OWASP? A OWASP (Open Web Application Security Project) é uma comunidade internacional sem fins lucrativos dedicada à segurança de aplicações(FOUNDATION, 2023). Ela publica guias, ferramentas e o famoso OWASP Top 10, que lista os dez riscos mais críticos de segurança para aplicações web, servindo como referência para desenvolvedores e empresas em todo o mundo.

- **Proteção contra injeção:** Injeção ocorre quando dados não confiáveis do usuário são enviados para um interpretador (como SQL ou comandos do sistema) como parte de um comando ou consulta(FOUNDATION, 2021a). Isso pode permitir que um atacante execute comandos maliciosos, como acessar, modificar ou excluir dados. No SecretarIA, prevenimos injeção validando e sanitizando todas as entradas, utilizando bibliotecas ORM como Prisma™ e Mongoose™, que geram consultas seguras e parametrizadas, além de nunca executar comandos diretamente a partir de dados do usuário.

- **Proteção contra exposição de dados sensíveis:** Exposição de dados sensíveis acontece quando informações confidenciais, como senhas, *tokens* ou dados pessoais, ficam acessíveis de forma não autorizada(FOUNDATION, 2021b). No SecretarIA, dados sensíveis são protegidos com criptografia (AES para informações confidenciais e bcrypt para senhas), e nunca são exibidos em texto claro. Além disso, as chaves da API e outros segredos são mantidos em variáveis de ambiente, fora do código-fonte, e o sistema utiliza HTTPS para proteger os dados em trânsito.

- **Proteção contra XSS (Cross-Site Scripting):** XSS é um tipo de ataque onde scripts maliciosos são inseridos em páginas web vistas por outros usuários, podendo roubar informações ou manipular a interface(FOUNDATION, 2021c). Para evitar XSS, o SecretarIA escapa e valida todas as entradas do usuário antes de exibi-las na interface, nunca permitindo que scripts sejam executados a partir

de campos de texto. Bibliotecas modernas de frontend como React™ já trazem proteções automáticas contra XSS, reforçando ainda mais a segurança.

- **Proteção contra falhas de controle de acesso:** Falhas de controle de acesso ocorrem quando usuários conseguem acessar recursos ou executar ações que não deveriam, como visualizar dados de outros usuários(FOUNDATION, 2021d). O SecretarIA implementa regras rigorosas de autenticação e autorização usando NextAuth™, garantindo que cada usuário só possa acessar seus próprios dados e funcionalidades permitidas. Todas as rotas e operações críticas são protegidas por verificações de permissões no backend.

Além dessas proteções, o sistema foi projetado em conformidade com a LGPD (LEI..., 2018), minimizando o envio de dados pessoais à IA e garantindo transparência ao usuário. O uso de HTTPS para todas as comunicações e a realização de testes específicos para identificar e mitigar vulnerabilidades reforçam a confiabilidade do SecretarIA.

4.1.15 Desafios, Aprendizados e Evolução

O desenvolvimento do SecretarIA foi marcado por desafios como a integração full-stack com Next.js, o gerenciamento eficiente de estado, a implementação segura de criptografia e a orquestração da IA. Cada obstáculo foi superado com pesquisa, testes e adaptação. Um dos maiores aprendizados foi a importância de documentar cada decisão e priorizar sempre a experiência do usuário.

Desafio 1: Integração Full-Stack com Next.js A curva de aprendizado do Next.js exigiu estudo aprofundado da documentação e exemplos práticos para conectar frontend e backend de forma eficiente.

Desafio 2: Gerenciamento de Estado Complexo Foi necessário estruturar o uso da Context API e hooks do React para manter o estado sincronizado entre módulos, especialmente em operações assíncronas.

Desafio 3: Criptografia Segura de Senhas A implementação do AES-256 no backend exigiu pesquisa sobre geração de chaves e IVs, além da integração segura com variáveis de ambiente.

Desafio 4: Integração com LangChain/OpenAI A orquestração do chat com IA demandou engenharia de prompt e tratamento cuidadoso das respostas para garantir precisão e segurança.

Desafio 5: Manutenção e Qualidade do Código Adoção de componentização, separação de responsabilidades e comentários detalhados foram essenciais para garantir manutenibilidade.

O projeto representa não apenas uma solução funcional, mas também uma trajetória de crescimento pessoal e técnico. Espero que este relato inspire outros desenvolvedores e contribua para o avanço de soluções inovadoras em assistentes pessoais inteligentes.

Outro aprendizado relevante foi a importância da gestão eficiente do tempo e da adaptação do escopo diante de imprevistos. A documentação contínua das decisões técnicas foi fundamental para manter a clareza e a qualidade do desenvolvimento.

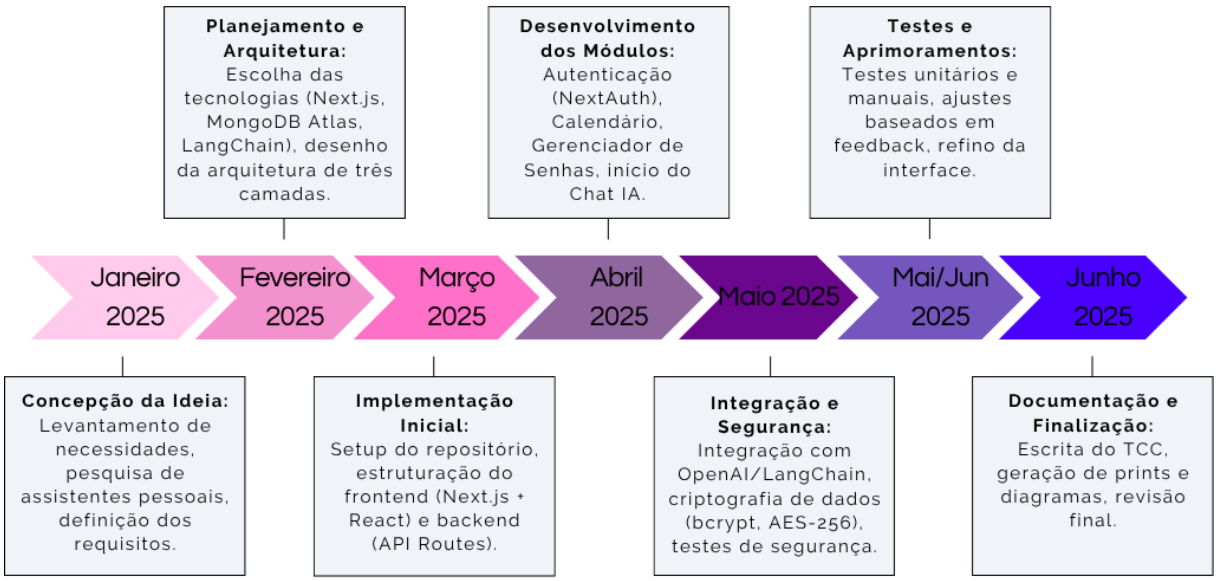


Figura 9 – Linha do tempo do desenvolvimento do SecretarIA

A linha do tempo destaca as principais etapas do desenvolvimento, evidenciando o planejamento, execução e aprimoramento contínuo do SecretarIA.

5 RESULTADOS

Este capítulo apresenta os resultados obtidos com o desenvolvimento do SecretarIA, discutindo criticamente suas funcionalidades, limitações e implicações práticas, bem como sugerindo possíveis aprimoramentos para versões futuras do sistema. A análise abrange a funcionalidade geral, a interação conversacional com IA, aspectos de usabilidade, segurança e privacidade.

5.0.1 Apresentação da Aplicação Final

Ao término do ciclo de desenvolvimento, o SecretarIA foi entregue como uma plataforma web acessível via navegador, integrando três módulos principais: Autenticação, Calendário e Gerenciador de Senhas, além de uma interface de Chat Inteligente para interação natural com a IA.

Interface Geral: A aplicação apresenta um layout moderno e responsivo, estilizado com Tailwind CSS™, e navegação intuitiva entre módulos após autenticação (Figuras 2 e 4). O design responsivo garante boa experiência em diferentes dispositivos.

Módulo Calendário: Permite visualização mensal, anual e em agenda dos eventos (Figuras 5, 2), com funcionalidades completas de CRUD (criar, editar, excluir) para eventos, tanto via interface gráfica quanto via chat.

Gerenciador de Senhas: Armazena credenciais de forma segura (Figuras 6), com criptografia AES-256 no backend e interface para adicionar, editar, ocultar e revelar senhas.

Chat Inteligente: Disponibiliza uma interface de conversação (Figura 7), onde o usuário interage em linguagem natural com a IA (GPT-3.5/4 via LangChain™) para consultar e manipular eventos do calendário, além de receber recomendações e resumos.

A arquitetura implementada utiliza Next.js™ para frontend e backend (API Routes), React™ para UI, MongoDB Atlas™ como banco de dados e integra a API da OpenAI™ via LangChain™ para processamento de linguagem natural.

5.0.2 Avaliação do Cumprimento dos Requisitos

A Tabela 3 resume o cumprimento dos requisitos funcionais definidos no Capítulo 4.

Tabela 3 – Avaliação dos Requisitos Funcionais

RF	Descrição	Status	Observações
RF01	Cadastro de Usuário	Cumprido	Implementado com e-mail/senha (bcrypt) e Google OAuth via NextAuth.js.
RF02	Login de Usuário	Cumprido	Autenticação segura implementada.
RF03	Recuperação de Senha	Cumprido	Recuperação via e-mail, com fluxo seguro.
RF04	Visualizar Calendário	Cumprido	Visualizações mês/ano/agenda disponíveis.
RF05	Adicionar Evento Manualmente	Cumprido	Via modal e API.
RF06	Editar Evento	Cumprido	Via modal e API.
RF07	Excluir Evento	Cumprido	Confirmação e API.
RF08	Listar Credenciais de Senha	Cumprido	Interface com senha mascarada por padrão.
RF09	Adicionar/Editar/Excluir Credencial de Senha	Cumprido	Formulário/modal e API, com criptografia backend.
RF10	Enviar Mensagem para IA (Chat)	Cumprido	Interface de chat implementada, integração backend.
RF11	Receber Resposta da IA (Chat)	Cumprido	Respostas exibidas na interface.
RF12	Consultar Agenda via Chat	Parcialmente Cumprido	IA responde a consultas simples, mas pode falhar em perguntas complexas.
RF13	Adicionar, Editar e Excluir Evento via Chat	Parcialmente Cumprido	IA interpreta comandos claros, mas pode errar em datas/horas complexas.

Como observado, as funcionalidades essenciais foram implementadas com sucesso. Os maiores desafios residem na robustez da interação com a IA (RF12 e RF13), discutidos a seguir.

5.0.3 Análise da Interação Conversacional com IA

A integração da IA via LangChain e OpenAI representa o núcleo inovador do SecretarIA, permitindo ao usuário interagir em linguagem natural para gerenciar sua

agenda. Os principais resultados observados foram:

Capacidades:

- Consultas Simples: A IA responde perguntas como “Quais meus compromissos de hoje?” ou “Tenho algo marcado para sexta-feira?”, recuperando eventos do MongoDB e apresentando respostas claras.
- Adição de Eventos Claros: Comandos como “Marcar reunião com equipe às 15h amanhã” são compreendidos, extraindo intenção, título, data e hora, e criando o evento.
- Manutenção de Contexto: Com uso de memória conversacional do LangChain, a IA consegue lembrar turnos anteriores para responder perguntas de acompanhamento.

Limitações e Desafios:

- Ambiguidade e Falta de Informação: Solicitações vagas (ex: “Marcar almoço semana que vem”) podem não ser corretamente interpretadas, faltando mecanismos de desambiguação.
- Extração de Entidades Complexas: Datas relativas complexas ou comandos ambíguos são frequentemente mal interpretados.
- Alucinações: O LLM pode gerar respostas plausíveis, porém incorretas, exigindo tratamento de erro no backend.
- Latência: O tempo de resposta da IA, devido à chamada à API da OpenAI, é maior que interações gráficas, impactando a experiência do usuário.
- Custo da API: O uso contínuo da OpenAI gera custos, limitando a escalabilidade do sistema.

Engenharia de Prompt: A experimentação com diferentes templates de prompt foi fundamental para orientar a IA. Instruções claras sobre o papel do assistente, formato das respostas e contexto estruturado da agenda foram essenciais para melhorar

a precisão.

Exemplo de Fluxo de Uso:

Usuário: *“Marque um evento para amanhã às 14h chamado Revisão do TCC”*
IA: *“Evento ’Revisão do TCC’ marcado para amanhã às 14h!”*

Apesar dos avanços, a complexidade da linguagem natural e as limitações dos LLMs atuais exigem abordagens mais sofisticadas para garantir robustez e precisão em todos os cenários.

5.0.4 Avaliação de Usabilidade

Com base nas heurísticas de Nielsen, a interface gráfica do SecretarIA apresenta:

- Visibilidade do Status: Indicadores de carregamento em chamadas de API e feedback visual no chat.
- Correspondência com o Mundo Real: Linguagem natural e design familiar.
- Controle e Liberdade: Navegação intuitiva entre módulos, com confirmações em operações sensíveis.
- Consistência: Design visual e navegação padronizados.
- Prevenção de Erros: Validações nos formulários; desafios permanecem na prevenção de erros de interpretação pela IA.
- Reconhecimento vs. Lembrança: Menus claros, mas o chat exige que o usuário conheça os comandos possíveis.
- Flexibilidade: UI gráfica eficiente para tarefas visuais; chat eficiente para comandos rápidos, limitado pelas capacidades da IA.
- Design Minimalista: Interface limpa, sem excesso de informação.
- Ajuda e Recuperação de Erros: Mensagens de erro básicas, especialmente no chat; documentação para o usuário ainda é limitada.

Conclui-se que a usabilidade da interface gráfica é boa, enquanto a interface conversacional herda os desafios inerentes à comunicação com LLMs, especialmente quanto à descoberta de funcionalidades e prevenção de erros de interpretação.

5.0.5 Discussão sobre Segurança e Privacidade

A segurança e privacidade foram prioridades desde o início do projeto:
Pontos Fortes:

- Uso de HTTPS para todas as comunicações.
- Autenticação robusta via NextAuth.js (hashing de senha, OAuth seguro).
- Controle de acesso rigoroso aos dados do usuário.
- Criptografia AES-256 para senhas no backend.
- Armazenamento seguro das chaves e segredos em variáveis de ambiente.

Áreas de Melhoria:

- Gestão da Chave de Criptografia: A derivação/armazenamento da chave AES poderia ser aprimorada, idealmente derivando-a da senha mestre do usuário.
- Privacidade dos Dados Enviados à OpenAI: Dados da agenda enviados para a API da OpenAI representam um risco inerente; alternativas incluem anonimização avançada ou uso de modelos locais.
- Segurança das Dependências: A atualização regular das bibliotecas utilizadas é fundamental para mitigar vulnerabilidades.

Em resumo, foram implementadas medidas sólidas de segurança, mas a privacidade na interação com IA externa e a gestão avançada de chaves permanecem como desafios para trabalhos futuros.

5.0.6 Considerações Finais dos Resultados

Os resultados obtidos demonstram que o SecretarIA cumpre os requisitos essenciais de um assistente pessoal inteligente, integrando módulos de agenda, senhas

e chat com IA em uma experiência unificada. O chat inteligente se destacou como diferencial, aproximando o sistema do conceito de assistente pessoal conversacional. As limitações observadas refletem desafios atuais da tecnologia, mas não comprometem a inovação e o potencial de evolução do sistema.

6 CONCLUSÃO

Este trabalho apresentou o desenvolvimento do SecretarIA, um assistente pessoal inteligente projetado para integrar funcionalidades de organização de agenda, gerenciamento seguro de senhas e interação conversacional via IA, buscando oferecer ao usuário uma experiência unificada, segura e inovadora. O projeto foi motivado pela crescente demanda por ferramentas de produtividade que conciliem praticidade, segurança e a capacidade de interação natural, aproveitando os avanços recentes em inteligência artificial generativa.

A implementação seguiu uma abordagem iterativa e incremental, utilizando uma stack tecnológica moderna baseada em Next.js (React/Node.js), MongoDB, LangChain e OpenAI, além de práticas consolidadas de segurança e usabilidade. Os módulos de autenticação, calendário, gerenciador de senhas e chat inteligente foram integrados em uma plataforma web responsiva, com foco tanto na experiência visual quanto na interação em linguagem natural.

A avaliação dos resultados demonstrou que os objetivos principais foram alcançados: o SecretarIA oferece uma plataforma funcional para o gerenciamento de compromissos e credenciais, permitindo ao usuário realizar operações tanto por meio de interfaces gráficas tradicionais quanto por comandos conversacionais no chat com IA. Destaca-se a centralidade do chat inteligente como diferencial do sistema, aproximando o SecretarIA do conceito de assistente pessoal conversacional e evidenciando o potencial dos LLMs para simplificar tarefas cotidianas.

No entanto, a experiência prática revelou limitações inerentes às tecnologias atuais de IA. A compreensão de comandos complexos, a extração precisa de entidades e a robustez na manutenção do contexto conversacional ainda apresentam desafios, especialmente em situações ambíguas ou que exigem raciocínio temporal avançado. Além disso, questões de privacidade e segurança permanecem relevantes, principalmente no que tange ao envio de dados sensíveis para APIs externas e à gestão das chaves de criptografia.

As contribuições deste trabalho incluem: (i) a demonstração prática da integração de LLMs com aplicações web full-stack, (ii) a análise crítica dos desafios de usabilidade, segurança e privacidade em assistentes pessoais inteligentes, e (iii) a sistematização de soluções e limitações encontradas durante o desenvolvimento, servindo de referência para futuros projetos na área.

Entre as principais limitações, destacam-se a ausência de testes formais com usuários finais, a dependência de APIs externas para processamento de linguagem natural, a necessidade de aprimorar a gestão de chaves criptográficas e a falta de funcionalidades avançadas como eventos recorrentes ou integrações com outros sistemas. O custo e a latência associados ao uso de LLMs em nuvem também impõem restrições para escalabilidade e uso em larga escala.

Como recomendações para trabalhos futuros, sugere-se: (i) aprimorar a robustez do chat inteligente por meio de agentes especializados e técnicas de desambiguação, (ii) realizar testes de usabilidade com usuários reais para validar e refinar a experiência conversacional, (iii) reforçar a segurança da gestão de senhas com derivação de chaves baseada na senha mestre do usuário, (iv) explorar o uso de modelos de IA open-source auto-hospedados para maior privacidade, e (v) expandir as funcionalidades da plataforma, incluindo eventos recorrentes, notificações inteligentes e integração com serviços externos como Google Calendar.

Em síntese, o SecretarIA representa um avanço na direção de assistentes pessoais inteligentes mais integrados e acessíveis, ao mesmo tempo em que evidencia os desafios técnicos e éticos envolvidos nessa jornada. Espera-se que as lições aprendidas e as soluções propostas neste trabalho possam contribuir para o desenvolvimento de sistemas cada vez mais úteis, seguros e alinhados às necessidades dos usuários na era da inteligência artificial.

REFERÊNCIAS

MAEDCHE, A.; LEGNER, C.; BENLIAN, A.; BERGER, B.; GIMPEL, H.; HESS, T.; HINZ, O.; MORANA, S.; SöLLNER, M. Ai-based digital assistants: Opportunities and challenges for organizations. **Business & Information Systems Engineering**, Springer, v. 61, n. 4, p. 535–544, 2019.

MOLS, A. et al. Household intelligent personal assistants in the netherlands: Exploring privacy concerns around surveillance, security, and platforms. **Journal of Information Security and Applications**, Elsevier, v. 58, p. 102718, 2021.

ISLAM, M. U. et al. Explainable artificial intelligence for cyber security. **IEEE Access**, v. 10, p. 45210–45229, 2022.

ISLAM, M. U.; CHAUDHRY, B. A framework to enhance user experience of older adults with speech-based intelligent personal assistants. **IEEE Access**, v. 11, p. 16683–16699, 2022.

ZHANG, R.; CAPRA, R.; MARCHIONINI, G. Personal information management: Challenges and opportunities. **Foundations and Trends in Information Retrieval**, Now Publishers, Inc., v. 13, n. 5, p. 418–518, 2019.

MAYER, S.; MüLLER, S.; SCHMIDT, A. Integration of ai-based personal assistants in everyday life: Challenges and perspectives. In: ACM. **Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems**. [S.l.], 2023. p. 1–13.

BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software Architecture in Practice**. [S.l.]: Addison-Wesley, 2012.

FOWLER, M. **Patterns of Enterprise Application Architecture**. [S.l.]: Addison-Wesley, 2002.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. **Design Patterns: Elements of Reusable Object-Oriented Software**. Boston, MA: Addison-Wesley, 1994. ISBN 978-0201633610.

VERCEL. **Next.js Documentation**. 2024. <https://nextjs.org/docs>.

PLATFORMS, I. M. **React Documentation**. 2024. <https://react.dev/>.

BOMMASANI, R.; HUDSON, D. A.; ADELI, E. et al. On the opportunities and risks of foundation models. **arXiv preprint arXiv:2108.07258**, 2021.

BROWN, T. B.; MANN, B.; RYDER, N. et al. Language models are few-shot learners. **Advances in Neural Information Processing Systems**, v. 33, p. 1877–1901, 2020.

LANGCHAIN. **LangChain Documentation**. 2024. <https://python.langchain.com/docs/>.

OPENAI. **OpenAI API Reference**. 2024. <https://platform.openai.com/docs>.

NEXTAUTH.JS. **NextAuth.js Documentation**. 2024. <https://next-auth.js.org/getting-started/introduction>.

NODEMAILER. **Nodemailer Documentation**. 2024. <https://nodemailer.com/about/>.

PRISMA. **Prisma Documentation**. 2024. <https://www.prisma.io/docs/>.

MONGOOSE. **Mongoose Documentation**. 2024. <https://mongoosejs.com/docs/>.

LABS, T. **Tailwind CSS Documentation**. 2024. <https://tailwindcss.com/docs>.

OWASP Foundation. **OWASP Top Ten 2021**. 2021. Disponível em: <https://owasp.org/www-project-top-ten/>.

BRASIL, P. da República do. **Lei Geral de Proteção de Dados Pessoais (LGPD) - Lei n.º 13.709, de 14 de agosto de 2018**. 2018. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm>. Acesso em: 27 jun. 2025.

PLATFORMS, I. M. **Jest Documentation**. 2024. <https://jestjs.io/docs/>.

LIBRARY, T. **Testing Library Documentation**. 2024. <https://testing-library.com/docs/>.

SCHWABER, K.; BEEDLE, M. **Agile Software Development with Scrum**. [S.l.]: Prentice Hall, 2002.

ANDERSON, D. J. **Kanban: Successful Evolutionary Change for Your Technology Business**. [S.l.]: Blue Hole Press, 2010.

ZHANG, Z.; ROSSI, R. A.; KVETON, B.; SHAO, Y.; YANG, D.; ZAMANI, H.; DERNONCOURT, F.; BARROW, J.; YU, T.; KIM, S.; ZHANG, R.; GU, J.; DERR, T.; CHEN, H.; WU, J.; CHEN, X.; WANG, Z.; MITRA, S.; LIPKA, N.; AHMED, N.; WANG, Y. Personalization of large language models: A survey. **arXiv preprint arXiv:2411.00027**, 2024. Disponível em: <https://arxiv.org/abs/2411.00027>.

KUMAR, P. Large language models (llms): survey, technical frameworks, and future challenges. **Artificial Intelligence Review**, Springer, v. 57, n. 10, p. 260, 2024.

ABDALKAREEM, R. A.; SHIHAB, E.; RILLING, J. Why do developers use trivial packages? an empirical case study on npm. In: ACM. **Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering**. [S.l.], 2017. p. 385–395.

RAUCH, G.; CONTRIBUTORS. Next.js: The react framework for production. In: **Next.js Documentation**. [s.n.], 2020. Disponível em: <https://nextjs.org/whitepaper.pdf>.

WATHAN, A.; SCHOGER, S.; REININK, J.; HEMPHILL, D. **Tailwind CSS: A Utility-First CSS Framework for Rapid UI Development**. [S.l.], 2020. Disponível em: <https://tailwindcss.com/docs/utility-first>. Disponível em: <https://tailwindcss.com/docs/utility-first>.

BANKER, K.; BAKKUM, P.; VERCH, S.; GARRETT, D.; HAWKINS, T. **MongoDB in Action**. 2nd. ed. [S.l.]: Manning Publications, 2016. Discussão sobre NoSQL, flexibilidade de esquema, integração com JavaScript e uso em nuvem.

PRISMA. **Prisma ORM Documentation**. [S.l.], 2022. Disponível em: <https://www.prisma.io/docs/>. Disponível em: <https://www.prisma.io/docs/>.

KARPOV, A. **Professional Node.js: Building Javascript Based Scalable Software**. [S.l.]: Apress, 2018. Inclui capítulo sobre modelagem e validação de dados com Mongoose.

CONTRIBUTORS, N. **NextAuth.js Documentation**. [S.l.], 2023. Disponível em: <https://next-auth.js.org/>. Disponível em: <https://next-auth.js.org/>.

CONTRIBUTORS, L. **LangChain Documentation**. [S.l.], 2023. Disponível em: <https://python.langchain.com/docs/>. Disponível em: <https://python.langchain.com/docs/>.

OPENAI. Gpt-4 technical report. **arXiv preprint arXiv:2303.08774**, 2023. Disponível em: <https://arxiv.org/abs/2303.08774>.

REINMAN, A. **Nodemailer: Easy as cake e-mail sending from your Node.js applications**. [S.l.], 2023. Disponível em: <https://nodemailer.com/about/>. Disponível em: <https://nodemailer.com/about/>.

MICROSOFT. **Visual Studio Code Documentation**. [S.l.], 2023. Disponível em: <https://code.visualstudio.com/docs>. Disponível em: <https://code.visualstudio.com/docs>.

CHACON, S.; STRAUB, B. **Pro Git**. 2nd. ed. [S.l.]: Apress, 2014. Disponível em: <https://git-scm.com/book/en/v2>.

GITHUB. **GitHub Docs**. [S.l.], 2023. Disponível em: <https://docs.github.com/>. Disponível em: <https://docs.github.com/>.

FOUNDATION, O. **About the OWASP Foundation**. [S.l.], 2023. Disponível em: <https://owasp.org/about/>. Disponível em: <https://owasp.org/about/>.

FOUNDATION, O. **OWASP Top 10:2021 - A03:2021 - Injection**. [S.l.], 2021. Disponível em: <https://owasp.org/Top10/A03_2021-Injection/>. Disponível em: <https://owasp.org/Top10/A03_2021-Injection/>.

FOUNDATION, O. **OWASP Top 10:2021 - A02:2021 - Cryptographic Failures**. [S.l.], 2021. Disponível em: <https://owasp.org/Top10/A02_2021-Cryptographic_Failures/>. Disponível em: <https://owasp.org/Top10/A02_2021-Cryptographic_Failures/>.

FOUNDATION, O. **OWASP Top 10:2021 - A07:2021 - Cross-Site Scripting (XSS)**. [S.l.], 2021. Disponível em: <https://owasp.org/Top10/A07_2021-Cross-Site_Scripting_(XSS)/>. Disponível em: <https://owasp.org/Top10/A07_2021-Cross-Site_Scripting_(XSS)/>.

FOUNDATION, O. **OWASP Top 10:2021 - A01:2021 - Broken Access Control**. [S.l.], 2021. Disponível em: <https://owasp.org/Top10/A01_2021-Broken_Access_Control/>. Disponível em: <https://owasp.org/Top10/A01_2021-Broken_Access_Control/>.

LEI Geral de Proteção de Dados Pessoais (LGPD) - Lei nº 13.709/2018. 2018. Brasil. Disponível em: <https://www.gov.br/secretariageral/pt-br/lgpd>.